

# Binary Search Tree & Balanced Trees — Complete Guide

---

**Target:** SDE Placements, Online Assessments, Competitive Programming

**Covers:** BST, AVL, Red-Black Tree, Treap, Ordered Set, Splay Tree, Persistent BST

---

## BST PROPERTY

---

### 1. Overview

A **Binary Search Tree (BST)** is a binary tree where every node satisfies:

- All nodes in the **left subtree** have values **less than** the node's value.
- All nodes in the **right subtree** have values **greater than** the node's value.
- Both left and right subtrees are themselves BSTs.

This property places a total order on the nodes, making BST the tree analogue of a sorted array.

### 2. Intuition

Think of the **telephone directory** or a **dictionary**:

- You open the book somewhere in the middle.
- If the word you want comes before the current page, you go left; otherwise, right.
- You repeat this, halving the search space each time.

The BST property is exactly this: **binary search on a tree structure**. At every node, the left subtree holds everything smaller, the right subtree holds everything larger. This lets you navigate to any value in  $O(h)$  comparisons, where  $h$  is the tree height.

### 3. When to Use It

- You need **dynamic ordering** — insertions, deletions, and searches interleaved.
- You need **sorted iteration** of elements.
- You need **order statistics** (floor, ceil, kth smallest).
- Problem statements mention "insert", "delete", "search", "sorted order", "successor", "predecessor".

### 4. When Not to Use It

- **Static data** — use a sorted array + binary search ( $O(\log n)$  search, no overhead).
- **Hash map suffices** — if you only need exact key lookup (no order, no range queries), `unordered_map` is  $O(1)$  average.
- **Data is already sorted** — inserting sorted data into a naive BST gives  $O(n)$  height (skewed tree). Use a balanced tree variant.
- **Tiny datasets** ( $< 50$  elements) — linear search in a vector may be faster due to cache locality.

## 5. Core Concepts

### 5.1 Node Structure

A BST node contains:

- **key / value** — the data.
- **left** — pointer to left child.
- **right** — pointer to right child.
- (optional) **parent** — pointer to parent (useful for deletion, successor).

### 5.2 Tree Height

Height = number of edges on the longest path from root to leaf. A balanced BST has height  $O(\log n)$ ; a skewed BST has height  $O(n)$ .

### 5.3 Recursive Substructure

Every subtree of a BST is also a BST. This allows recursive algorithms for search, insert, and delete.

### 5.4 Inorder Traversal

Visiting left → root → right yields sorted order. This is a **direct consequence** of the BST property.

## 6. Step-by-Step Algorithm (Generic BST Operations)

Covered in dedicated sections below.

## 7. Dry Run

Covered in dedicated sections below.

## 8. C++ Implementation (Generic BST Node)

```
#include <bits/stdc++.h>
using namespace std;

template <typename T>
struct BSTNode {
    T data;
    BSTNode *left, *right;
    BSTNode(T val) : data(val), left(nullptr), right(nullptr) {}
};
```

## 9. Python Implementation (Generic BST Node)

```
class BSTNode:
    def __init__(self, val):
        self.data = val
```

```
self.left = None
self.right = None
```

## 10. Code Explanation

The node stores the value and two child pointers. The template/type parameter allows reusability for `int`, `long long`, `string`, or custom comparable types.

## 11. Complexity Analysis

| Operation | Average (Balanced) | Worst (Skewed) |
|-----------|--------------------|----------------|
| Search    | $O(\log n)$        | $O(n)$         |
| Insert    | $O(\log n)$        | $O(n)$         |
| Delete    | $O(\log n)$        | $O(n)$         |
| Space     | $O(n)$             | $O(n)$         |

# SEARCH IN BST

## 1. Overview

Searching in a BST finds whether a key exists in the tree. It exploits the BST property to decide the direction at each node.

## 2. Intuition

**Book search analogy:** You open the book. If the word is on the current page, done. If it's smaller, go to the left half; if larger, go to the right half. Repeat.

At each node, you compare the target with the current node's value:

- Equal → found.
- Smaller → go left.
- Larger → go right.
- Null → not found.

## 3. When to Use It

- You need to check membership in a dynamic set.
- You need to locate a node for subsequent operations (floor, ceil, delete, successor).

## 4. When Not to Use It

- Only membership check, no order needed → `unordered_set`  $O(1)$  average.
- Only a few searches on static data → sort + binary search.

## 5. Core Concepts

### 5.1 Comparison Direction

At each node, the BST property tells you exactly which subtree can contain the key. This is what makes search efficient.

### 5.2 Termination

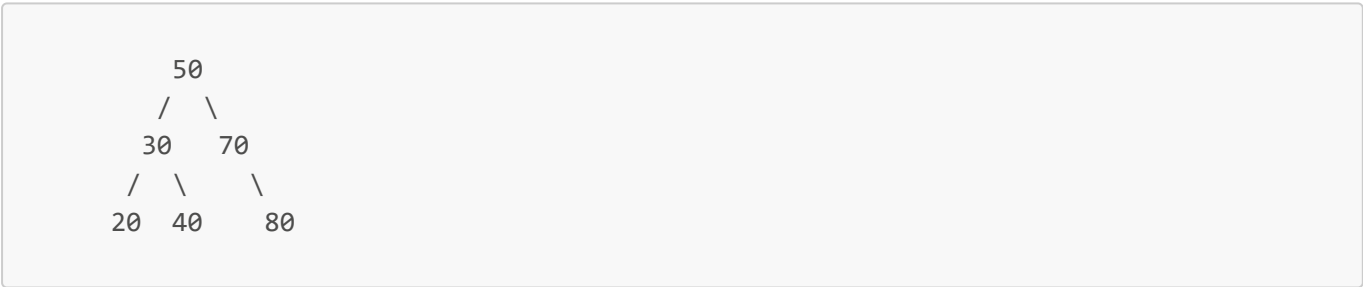
If you reach `nullptr`, the key is definitely not present.

## 6. Step-by-Step Algorithm

```
function search(root, key):
    if root == null      -> return false
    if root.data == key  -> return true
    if key < root.data   -> return search(root.left, key)
    else                -> return search(root.right, key)
```

## 7. Dry Run

Tree:



Search for 40:

| Step | Node | Comparison | Action   |
|------|------|------------|----------|
| 1    | 50   | 40 < 50    | Go left  |
| 2    | 30   | 40 > 30    | Go right |
| 3    | 40   | 40 == 40   | Found! ✓ |

Search for 100:

| Step | Node | Comparison | Action      |
|------|------|------------|-------------|
| 1    | 50   | 100 > 50   | Go right    |
| 2    | 70   | 100 > 70   | Go right    |
| 3    | 80   | 100 > 80   | Go right    |
| 4    | null | —          | Not found ✗ |

## 8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

template <typename T>
struct Node {
    T data;
    Node *left, *right;
    Node(T val) : data(val), left(nullptr), right(nullptr) {}
};

// --- Iterative Search (preferred for CP, avoids recursion depth) ---
template <typename T>
bool searchBST(Node<T>* root, T key) {
    while (root) {
        if (root->data == key) return true;
        root = (key < root->data) ? root->left : root->right;
    }
    return false;
}

// --- Recursive Search ---
template <typename T>
bool searchBSTRec(Node<T>* root, T key) {
    if (!root) return false;
    if (root->data == key) return true;
    if (key < root->data) return searchBSTRec(root->left, key);
    return searchBSTRec(root->right, key);
}
```

## 9. Python Implementation

```
class Node:
    def __init__(self, val):
        self.data = val
        self.left = None
        self.right = None

def search_bst(root, key):
    """Iterative search."""
    cur = root
    while cur:
        if cur.data == key:
            return True
        cur = cur.left if key < cur.data else cur.right
    return False

def search_bst_rec(root, key):
    """Recursive search."""
```

```
if not root:
    return False
if root.data == key:
    return True
if key < root.data:
    return search_bst_rec(root.left, key)
return search_bst_rec(root.right, key)
```

## 10. Code Explanation

- **Iterative version:** Uses a `while` loop; at each step goes left or right. No recursion overhead. Preferred for CP.
- **Recursive version:** Cleaner but may cause stack overflow on deep trees in languages without tail-call optimization.
- Both run in  $O(h)$  time.

## 11. Complexity Analysis

| Case           | Time        | Space (iterative) | Space (recursive) |
|----------------|-------------|-------------------|-------------------|
| Best (root)    | $O(1)$      | $O(1)$            | $O(1)$            |
| Average        | $O(\log n)$ | $O(1)$            | $O(\log n)$       |
| Worst (skewed) | $O(n)$      | $O(1)$            | $O(n)$            |

## 12. Common Patterns

- **Binary search on tree:** Same as BST search but with custom comparison.
- **Range query:** Search for lower bound, then traverse.

## 13. Common Mistakes

- Forgetting to handle `nullptr`.
- Comparing wrong direction (go left when key is larger).
- Using recursion without considering stack overflow on skewed trees.

## 14. Edge Cases

| Case             | Expected                                    |
|------------------|---------------------------------------------|
| Empty tree       | false                                       |
| Key at root      | true                                        |
| Key not present  | false                                       |
| Duplicate values | depends on convention (usually not allowed) |

## 15. Variations

- **Search for closest value** — track min difference while traversing.
- **Search with parent tracking** — return the node and its parent (used in deletion).

## 16. Related Algorithms

- **Binary search on array** — same idea, different data structure.
- **Lower bound / upper bound** — BST search variant.

## 17. Practice Problems

### Easy:

- [Search in a Binary Search Tree — LeetCode 700](#)
- [Search a node in BST — GFG](#)

### Medium:

- [Closest Binary Search Tree Value — LeetCode 270](#)
- [Validate Binary Search Tree — LeetCode 98](#) (uses search property)

## 18. Interview Explanation

"BST search works by comparing the target with the current node. If equal, we found it. If smaller, we go left; if larger, we go right. We continue until we find the key or hit null. The BST property guarantees correctness, and the time is  $O(h)$  which is  $O(\log n)$  on average."

## 19. Revision Notes

- Start at root, compare, go left/right.
- Return `false` on null.
- Iterative preferred to avoid recursion depth issues.

## 20. Final Cheat Sheet

| Aspect     | Detail                                |
|------------|---------------------------------------|
| When       | Dynamic set membership with order     |
| How        | Compare → go left/right               |
| Complexity | $O(h)$ time, $O(1)$ space (iterative) |
| Edge       | Empty tree, missing key               |

# INSERT IN BST

## 1. Overview

Insertion adds a new key to the BST while maintaining the BST property. The new node is always inserted as a leaf.

## 2. Intuition

**Adding a word to a dictionary:** You find the correct page and position, then insert the word. In a BST, you search for the key; when you hit a null pointer, you put the new node there.

The BST property is preserved because you followed the BST search path to find the correct insertion point.

## 3. When to Use It

- Building a BST from a stream of elements.
- Adding elements to an ordered set dynamically.
- Along with search and delete in a dynamic ordered container.

## 4. When Not to Use It

- Inserting many elements at once — use **sort + build balanced BST**  $O(n \log n)$  instead of  $n$  insertions  $O(n^2)$  worst).
- Inserting sorted data into naive BST — creates a skewed tree. Use a balanced BST variant.

## 5. Core Concepts

### 5.1 Insertion Point

The new node is always placed at the position where the search would have failed. This is the **leaf position** in the correct path.

### 5.2 Duplicate Handling

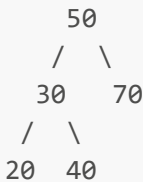
Convention: BSTs typically do **not** allow duplicates. If a duplicate is inserted, you can either ignore it or increment a frequency counter.

## 6. Step-by-Step Algorithm

```
function insert(root, key):
    if root == null:
        return new Node(key)
    if key < root.data:
        root.left = insert(root.left, key)
    else if key > root.data:
        root.right = insert(root.right, key)
    // else: duplicate, do nothing (or increment count)
    return root
```

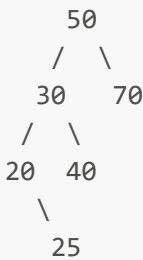
## 7. Dry Run

**Insert 25 into:**



| Step | Node | key < node? | Action         |
|------|------|-------------|----------------|
| 1    | 50   | 25 < 50     | Go left        |
| 2    | 30   | 25 < 30     | Go left        |
| 3    | 20   | 25 > 20     | Go right       |
| 4    | null | —           | Insert 25 here |

Result:



## 8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

template <typename T>
struct Node {
    T data;
    Node *left, *right;
    Node(T val) : data(val), left(nullptr), right(nullptr) {}
};

// --- Recursive Insert ---
template <typename T>
Node<T>* insertBST(Node<T>* root, T key) {
    if (!root) return new Node<T>(key);
    if (key < root->data)
        root->left = insertBST(root->left, key);
    else if (key > root->data)
        root->right = insertBST(root->right, key);
    // duplicate: ignored
    return root;
}
```

```

}

// --- Iterative Insert (avoids recursion depth) ---
template <typename T>
Node<T>* insertBSTIter(Node<T>* root, T key) {
    Node<T>* newNode = new Node<T>(key);
    if (!root) return newNode;

    Node<T>* cur = root;
    Node<T>* parent = nullptr;
    while (cur) {
        parent = cur;
        if (key < cur->data)
            cur = cur->left;
        else if (key > cur->data)
            cur = cur->right;
        else
            return root; // duplicate
    }
    if (key < parent->data)
        parent->left = newNode;
    else
        parent->right = newNode;
    return root;
}

```

## 9. Python Implementation

```

class Node:
    def __init__(self, val):
        self.data = val
        self.left = None
        self.right = None

def insert_bst(root, key):
    """Recursive insert."""
    if not root:
        return Node(key)
    if key < root.data:
        root.left = insert_bst(root.left, key)
    elif key > root.data:
        root.right = insert_bst(root.right, key)
    return root

def insert_bst_iter(root, key):
    """Iterative insert."""
    new_node = Node(key)
    if not root:
        return new_node
    cur, parent = root, None
    while cur:

```

```

    parent = cur
    if key < cur.data:
        cur = cur.left
    elif key > cur.data:
        cur = cur.right
    else:
        return root # duplicate
    if key < parent.data:
        parent.left = new_node
    else:
        parent.right = new_node
    return root

```

## 10. Code Explanation

- **Recursive:** Base case creates a new node. Recursively goes left or right. Returns the (possibly new) root of each subtree.
- **Iterative:** Uses `parent` pointer to track where to attach the new node. Avoids recursion depth issues.
- Duplicates are silently ignored.

## 11. Complexity Analysis

| Case           | Time        | Space                                    |
|----------------|-------------|------------------------------------------|
| Average        | $O(\log n)$ | $O(\log n)$ recursive / $O(1)$ iterative |
| Worst (skewed) | $O(n)$      | $O(n)$ recursive / $O(1)$ iterative      |

## 12. Common Patterns

- **Building BST from array:** Iterate and insert each element.
- **Building balanced BST from sorted array:** Pick middle as root, recursively build left and right halves.

## 13. Common Mistakes

- Not updating the parent's child pointer.
- Forgetting to return `root` at the end of recursive insert.
- Not handling duplicates.

## 14. Edge Cases

| Case                    | Expected               |
|-------------------------|------------------------|
| Empty tree              | New node becomes root  |
| Insert smaller than all | Becomes leftmost leaf  |
| Insert larger than all  | Becomes rightmost leaf |
| Duplicate               | No change              |

## 15. Variations

- **Insert with parent pointer** — useful for deletion.
- **Insert with frequency count** — allow duplicates by counting.

## 16. Practice Problems

### Easy:

- [Insert into a Binary Search Tree — LeetCode 701](#)
- [Binary Search Tree Insertion — GFG](#)

### Medium:

- [Construct BST from Preorder — LeetCode 1008](#)

## 17. Interview Explanation

"To insert in a BST, we first search for the key. When we reach a null pointer, we insert the new node there. The recursive version is clean but the iterative version avoids stack overflow. We ignore duplicates by convention."

## 18. Revision Notes

- Insert always at leaf position.
- Recursive: base case = new node, return root at end.
- Iterative: track parent, attach after loop.
- Duplicates → ignored.

## 19. Final Cheat Sheet

| Aspect     | Detail                       |
|------------|------------------------------|
| When       | Dynamic ordered set building |
| How        | Search → insert at null      |
| Complexity | $O(h)$ time                  |
| Edge       | Empty tree, duplicates       |

## DELETE IN BST

### 1. Overview

Deletion removes a node from the BST while preserving the BST property. It is the most complex BST operation because the node to delete may have 0, 1, or 2 children, each requiring different handling.

### 2. Intuition

### Removing a page from a dictionary:

- If the page is a leaf (no sub-entries), just remove it.
- If it has one sub-section, replace it with that sub-section.
- If it has two sub-sections, find the next page in order (the smallest page in the right section), replace the current page's content with that, and delete that next page instead.

The key insight: **the inorder successor (or predecessor) always has at most one child**, making it easier to delete.

## 3. When to Use It

- Maintaining a dynamic ordered set with removals.
- Implementing ordered maps, priority queues with deletion.
- Database indexing operations.

## 4. When Not to Use It

- Frequent deletions with only a few elements → **vector** + **erase-remove** idiom.
- Only deletions, no order needed → **unordered\_set**.
- Need to delete range → use a balanced BST with split/merge (Treap).

## 5. Core Concepts

### 5.1 Three Cases

1. **Leaf (0 children)**: Simply remove the node (set parent's pointer to null).
2. **One child**: Replace the node with its child.
3. **Two children**: Find the inorder successor (smallest in right subtree), copy its value to the node, then delete the successor (which falls into case 1 or 2).

### 5.2 Inorder Successor

The smallest node in the right subtree. It is guaranteed to have at most one child (no left child), making it easy to delete.

### 5.3 Inorder Predecessor

Largest node in the left subtree. Can also be used as an alternative to the successor.

## 6. Step-by-Step Algorithm

```
function delete(root, key):
    if root == null: return null

    if key < root.data:
        root.left = delete(root.left, key)
    else if key > root.data:
        root.right = delete(root.right, key)
    else:
```

```

// Case 1: leaf
if root.left == null and root.right == null:
    delete root
    return null

// Case 2: one child
if root.left == null:
    Node* temp = root.right
    delete root
    return temp
if root.right == null:
    Node* temp = root.left
    delete root
    return temp

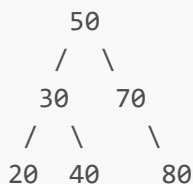
// Case 3: two children
Node* succ = inorderSuccessor(root.right)
root.data = succ.data
root.right = delete(root.right, succ.data)

return root

```

## 7. Dry Run

**Delete 50 from:**



**Step 1:** Found 50 (root). Two children.

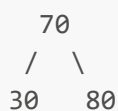
**Step 2:** Find inorder successor of 50 = smallest in right subtree = 70.

**Step 3:** Copy 70 to root. Now delete 70 from right subtree.

**Step 4:** Delete 70 from right subtree:

- 70 has one child (80).
- Replace 70 with 80.

**Result:**



```

  /  \
20   40

```

**Delete 20 from:**

```

      50
     /  \
    30   70
   /  \
  20   40

```

**Step 1:** Found 20 (leaf).

**Step 2:** Delete it, return null.

**Result:**

```

      50
     /  \
    30   70
     \
      40

```

## 8. C++ Implementation

```

#include <bits/stdc++.h>
using namespace std;

template <typename T>
struct Node {
    T data;
    Node *left, *right;
    Node(T val) : data(val), left(nullptr), right(nullptr) {}
};

// Helper: find inorder successor (smallest in right subtree)
template <typename T>
Node<T>* findMin(Node<T>* root) {
    while (root && root->left) root = root->left;
    return root;
}

// Delete a key from BST
template <typename T>
Node<T>* deleteBST(Node<T>* root, T key) {
    if (!root) return nullptr;

```

```

if (key < root->data) {
    root->left = deleteBST(root->left, key);
} else if (key > root->data) {
    root->right = deleteBST(root->right, key);
} else {
    // Found the node to delete
    // Case 1: Leaf
    if (!root->left && !root->right) {
        delete root;
        return nullptr;
    }
    // Case 2: One child
    if (!root->left) {
        Node<T>* temp = root->right;
        delete root;
        return temp;
    }
    if (!root->right) {
        Node<T>* temp = root->left;
        delete root;
        return temp;
    }
    // Case 3: Two children
    Node<T>* succ = findMin(root->right);
    root->data = succ->data;
    root->right = deleteBST(root->right, succ->data);
}
return root;
}

```

## 9. Python Implementation

```

class Node:
    def __init__(self, val):
        self.data = val
        self.left = None
        self.right = None

def find_min(root):
    while root and root.left:
        root = root.left
    return root

def delete_bst(root, key):
    if not root:
        return None

    if key < root.data:
        root.left = delete_bst(root.left, key)
    elif key > root.data:

```

```
        root.right = delete_bst(root.right, key)
    else:
        # Case 1: Leaf
        if not root.left and not root.right:
            return None
        # Case 2: One child
        if not root.left:
            return root.right
        if not root.right:
            return root.left
        # Case 3: Two children
        succ = find_min(root.right)
        root.data = succ.data
        root.right = delete_bst(root.right, succ.data)
    return root
```

## 10. Code Explanation

- **Search phase:** Recursively find the node to delete, updating parent pointers.
- **Delete phase:** Handle the three cases.
- **Two children case:** Copy successor's value, then recursively delete the successor. This avoids complex pointer manipulation.
- The `findMin` helper goes to the leftmost node in a subtree.

## 11. Complexity Analysis

| Case           | Time                        | Space  |
|----------------|-----------------------------|--------|
| Leaf           | $O(h)$                      | $O(h)$ |
| One child      | $O(h)$                      | $O(h)$ |
| Two children   | $O(h) + O(h)$ for successor | $O(h)$ |
| Worst (skewed) | $O(n)$                      | $O(n)$ |

## 12. Common Patterns

- **Delete with parent pointer** — avoids recursion by tracking parent.
- **Merge deletion** — instead of copying value, merge left and right subtrees (used in some implementations).

## 13. Common Mistakes

- Forgetting to free memory (C++).
- Not handling the case where root is the node to delete.
- Incorrect successor finding.
- Memory leak: losing child pointers when deleting a node with children.

## 14. Edge Cases

| Case                       | Expected                      |
|----------------------------|-------------------------------|
| Delete from empty tree     | nullptr                       |
| Delete root (leaf)         | nullptr                       |
| Delete root (one child)    | root's child becomes new root |
| Delete root (two children) | successor becomes new root    |
| Delete non-existent key    | no change                     |
| Only one node in tree      | nullptr                       |

## 15. Variations

- **Lazy deletion:** Mark node as deleted without removing it. Simpler but wastes space.
- **Delete with predecessor:** Use inorder predecessor instead of successor.

## 16. Practice Problems

### Easy:

- [Delete Node in a BST — LeetCode 450](#)
- [Delete a node from BST — GFG](#)

### Medium:

- [Remove BST Keys Outside Given Range — GFG](#)

## 17. Interview Explanation

"BST deletion has three cases. If the node is a leaf, we just remove it. If it has one child, we replace it with that child. If it has two children, we find the inorder successor (smallest in the right subtree), copy its value, and delete the successor. The successor always has at most one child, so it's easy to delete."

## 18. Revision Notes

- 3 cases: leaf, one child, two children.
- Two children → use successor (or predecessor).
- Successor = min in right subtree.
- Recursive deletion of successor avoids complex pointer manipulation.

## 19. Final Cheat Sheet

| Aspect       | Detail                            |
|--------------|-----------------------------------|
| When         | Dynamic ordered set with removals |
| Cases        | 0, 1, or 2 children               |
| Two children | Copy successor, delete successor  |
| Complexity   | O(h) time                         |

---

# INORDER GIVES SORTED ORDER

---

## 1. Overview

Inorder traversal of a BST visits nodes in **non-decreasing order**. This is a direct consequence of the BST property and is the fundamental reason BSTs are used for ordered data.

## 2. Intuition

The BST property says:  $\text{left} < \text{root} < \text{right}$ . Inorder traversal visits  $\text{left} \rightarrow \text{root} \rightarrow \text{right}$ . So you visit all smaller elements first, then the current node, then all larger elements. This naturally produces sorted order.

**Analogy:** Reading a dictionary page by page in order — you start from the first page (leftmost) and go to the last (rightmost).

## 3. When to Use It

- Getting sorted output from a BST.
- Validating whether a tree is a BST (inorder must be sorted).
- Converting BST to sorted array / linked list.
- Merging two BSTs (merge their inorder traversals).

## 4. When Not to Use It

- You only need to know if the tree is sorted — a range check during traversal is more efficient.
- You need to maintain sorted order with modifications — use a balanced BST with inorder tracking.

## 5. Core Concepts

### 5.1 Traversal Order

```
inorder(node):  
    inorder(node.left)  
    visit(node)  
    inorder(node.right)
```

### 5.2 Strictly Increasing?

If duplicates are not allowed, inorder is strictly increasing. If duplicates are allowed, it's non-decreasing.

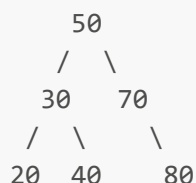
## 6. Step-by-Step Algorithm

```
function inorder(root):  
    if root == null: return  
    inorder(root.left)
```

```
print(root.data)
inorder(root.right)
```

## 7. Dry Run

**Tree:**



**Inorder traversal steps:**

1. Go to 50 → go left to 30 → go left to 20 → go left (null) → **visit 20** → go right (null) → return to 30 → **visit 30** → go right to 40 → go left (null) → **visit 40** → return to 30 → return to 50 → **visit 50** → go right to 70 → go left (null) → **visit 70** → go right to 80 → go left (null) → **visit 80**.

**Output:** 20, 30, 40, 50, 70, 80 ✅ (sorted)

## 8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

template <typename T>
struct Node {
    T data;
    Node *left, *right;
    Node(T val) : data(val), left(nullptr), right(nullptr) {}
};

// Recursive inorder
template <typename T>
void inorder(Node<T>* root) {
    if (!root) return;
    inorder(root->left);
    cout << root->data << " ";
    inorder(root->right);
}

// Iterative inorder (Morris / stack-based)
// Stack-based (no parent pointer modification)
template <typename T>
void inorderIterative(Node<T>* root) {
    stack<Node<T>*> st;
    Node<T>* cur = root;
    while (cur || !st.empty()) {
```

```

        while (cur) {
            st.push(cur);
            cur = cur->left;
        }
        cur = st.top(); st.pop();
        cout << cur->data << " ";
        cur = cur->right;
    }
}

// Morris inorder (O(1) space, modifies tree temporarily)
template <typename T>
void inorderMorris(Node<T>* root) {
    Node<T>* cur = root;
    while (cur) {
        if (!cur->left) {
            cout << cur->data << " ";
            cur = cur->right;
        } else {
            Node<T>* pred = cur->left;
            while (pred->right && pred->right != cur)
                pred = pred->right;
            if (!pred->right) {
                pred->right = cur; // thread
                cur = cur->left;
            } else {
                pred->right = nullptr; // remove thread
                cout << cur->data << " ";
                cur = cur->right;
            }
        }
    }
}

```

## 9. Python Implementation

```

class Node:
    def __init__(self, val):
        self.data = val
        self.left = None
        self.right = None

def inorder(root):
    """Recursive inorder."""
    if not root:
        return
    inorder(root.left)
    print(root.data, end=" ")
    inorder(root.right)

def inorder_iterative(root):

```

```
"""Stack-based iterative inorder."""
stack, cur = [], root
while cur or stack:
    while cur:
        stack.append(cur)
        cur = cur.left
    cur = stack.pop()
    print(cur.data, end=" ")
    cur = cur.right
```

## 10. Code Explanation

- **Recursive:** Cleanest. Uses call stack. Risk of overflow on deep trees.
- **Iterative (stack):** Explicit stack. Avoids recursion depth issues. Standard for CP.
- **Morris:** O(1) space by creating temporary threads (right pointers to parent). Modifies tree during traversal; restores it afterwards.

## 11. Complexity Analysis

| Version           | Time | Space           |
|-------------------|------|-----------------|
| Recursive         | O(n) | O(h) call stack |
| Iterative (stack) | O(n) | O(h) stack      |
| Morris            | O(n) | O(1)            |

## 12. Common Patterns

- **Validate BST:** Check if inorder is strictly increasing.
- **Kth smallest:** Stop inorder at kth element.
- **BST to sorted array:** Store inorder traversal.
- **Merge two BSTs:** Merge their inorder traversals (like merge sort).

## 13. Common Mistakes

- Confusing inorder with preorder/postorder.
- Not handling null nodes.
- Using Morris traversal in a multithreaded environment (tree modification).

## 14. Edge Cases

| Case         | Expected         |
|--------------|------------------|
| Empty tree   | Nothing printed  |
| Single node  | Print that node  |
| Skewed left  | Output is sorted |
| Skewed right | Output is sorted |

## 15. Variations

- **Preorder:** root → left → right (used for tree copy, serialization).
- **Postorder:** left → right → root (used for deletion, expression trees).
- **Level order:** BFS traversal.

## 16. Practice Problems

### Easy:

- [Binary Tree Inorder Traversal — LeetCode 94](#)
- [Inorder Traversal — GFG](#)

### Medium:

- [Validate Binary Search Tree — LeetCode 98](#)
- [Kth Smallest Element in a BST — LeetCode 230](#)

## 17. Interview Explanation

"Inorder traversal of a BST visits nodes in sorted order because of the BST property: left < root < right. The traversal visits left subtree first, then root, then right subtree. This is the basis for many BST algorithms like validation, kth smallest, and range queries."

## 18. Revision Notes

- Inorder = left → root → right = sorted.
- Recursive is simplest, iterative is safer for deep trees.
- Morris gives O(1) space but modifies tree.

## 19. Final Cheat Sheet

| Aspect     | Detail                            |
|------------|-----------------------------------|
| Order      | left → root → right               |
| Property   | Produces sorted order             |
| Complexity | O(n) time                         |
| Space      | O(h) recursive/stack, O(1) Morris |

---

## FLOOR AND CEIL IN BST

---

### 1. Overview

**Floor:** Largest value in BST  $\leq$  given key.

**Ceil:** Smallest value in BST  $\geq$  given key.

These are the BST analogues of `lower_bound` and `upper_bound` in sorted arrays.

## 2. Intuition

**Finding a parking spot:** You want the spot closest to the entrance but not beyond the entrance (floor). Or the spot closest to the entrance but not before it (ceil).

In a BST:

- **Floor:** As you go down, whenever you see a node  $\leq$  key, it's a candidate. Try to go right for a larger (but still  $\leq$  key) value.
- **Ceil:** Whenever you see a node  $\geq$  key, it's a candidate. Try to go left for a smaller (but still  $\geq$  key) value.

## 3. When to Use It

- Finding predecessor/successor of a key in a dynamic set.
- Range queries: find all elements in  $[L, R]$  by finding floor(L) and then iterating.
- Problems: "nearest element", "closest value", "just greater", "just smaller".

## 4. When Not to Use It

- Static data  $\rightarrow$  use sorted array + binary search (lower\_bound/upper\_bound).
- Only need exact match  $\rightarrow$  use search.
- Multiple queries on static data  $\rightarrow$  preprocess into sorted array.

## 5. Core Concepts

### 5.1 Floor vs Predecessor

- **Floor of key:** largest  $\leq$  key. If key exists, floor = key itself.
- **Predecessor of key** (strictly smaller): largest  $<$  key. Equivalent to floor when key is not present.

### 5.2 Ceil vs Successor

- **Ceil of key:** smallest  $\geq$  key. If key exists, ceil = key itself.
- **Successor** (strictly larger): smallest  $>$  key.

## 6. Step-by-Step Algorithm

**Floor:**

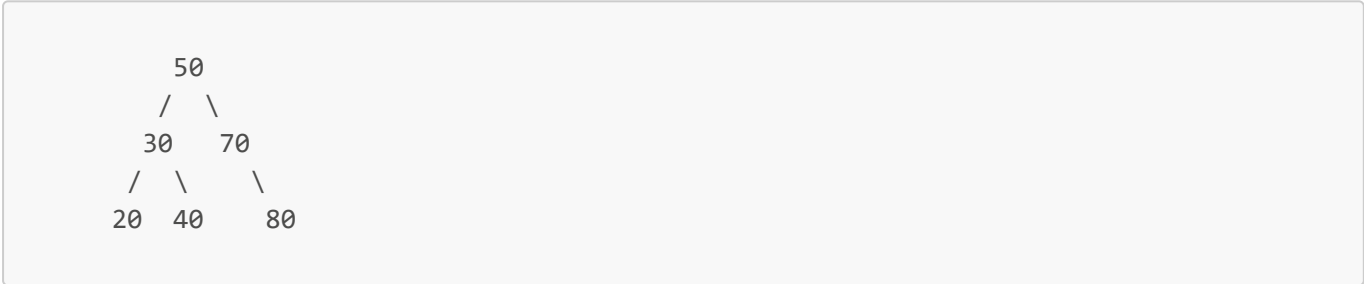
```
function floor(root, key):
    ans = -inf
    while root:
        if root.data == key: return key
        if root.data < key:
            ans = root.data    // candidate
            root = root.right  // try for larger
        else:
            root = root.left   // too big, go left
    return ans
```

Ceil:

```
function ceil(root, key):
    ans = +inf
    while root:
        if root.data == key: return key
        if root.data > key:
            ans = root.data    // candidate
            root = root.left   // try for smaller
        else:
            root = root.right  // too small, go right
    return ans
```

7. Dry Run

Tree:



Floor of 55:

| Step | Node | Condition | ans | Next  |
|------|------|-----------|-----|-------|
| 1    | 50   | 50 < 55   | 50  | right |
| 2    | 70   | 70 > 55   | 50  | left  |
| 3    | null | —         | 50  | done  |

Result: 50

Floor of 65:

| Step | Node | Condition | ans | Next  |
|------|------|-----------|-----|-------|
| 1    | 50   | 50 < 65   | 50  | right |
| 2    | 70   | 70 > 65   | 50  | left  |
| 3    | null | —         | 50  | done  |

Result: 50

Ceil of 55:

| Step | Node | Condition | ans      | Next  |
|------|------|-----------|----------|-------|
| 1    | 50   | $50 < 55$ | $\infty$ | right |
| 2    | 70   | $70 > 55$ | 70       | left  |
| 3    | null | —         | 70       | done  |

**Result:** 70

**Ceil of 65:**

| Step | Node | Condition | ans      | Next  |
|------|------|-----------|----------|-------|
| 1    | 50   | $50 < 65$ | $\infty$ | right |
| 2    | 70   | $70 > 65$ | 70       | left  |
| 3    | null | —         | 70       | done  |

**Result:** 70

## 8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

template <typename T>
struct Node {
    T data;
    Node *left, *right;
    Node(T val) : data(val), left(nullptr), right(nullptr) {}
};

// Floor: largest value <= key
template <typename T>
T floorBST(Node<T>* root, T key) {
    T ans = numeric_limits<T>::min();
    while (root) {
        if (root->data == key) return key;
        if (root->data < key) {
            ans = root->data;
            root = root->right;
        } else {
            root = root->left;
        }
    }
    return ans;
}

// Ceil: smallest value >= key
template <typename T>
T ceilBST(Node<T>* root, T key) {
```

```

    T ans = numeric_limits<T>::max();
    while (root) {
        if (root->data == key) return key;
        if (root->data > key) {
            ans = root->data;
            root = root->left;
        } else {
            root = root->right;
        }
    }
    return ans;
}

// Successor: smallest value > key (strict)
template <typename T>
T successorBST(Node<T>* root, T key) {
    T ans = numeric_limits<T>::max();
    while (root) {
        if (root->data > key) {
            ans = root->data;
            root = root->left;
        } else {
            root = root->right;
        }
    }
    return ans;
}

// Predecessor: largest value < key (strict)
template <typename T>
T predecessorBST(Node<T>* root, T key) {
    T ans = numeric_limits<T>::min();
    while (root) {
        if (root->data < key) {
            ans = root->data;
            root = root->right;
        } else {
            root = root->left;
        }
    }
    return ans;
}

```

## 9. Python Implementation

```

class Node:
    def __init__(self, val):
        self.data = val
        self.left = None
        self.right = None

```

```
def floor_bst(root, key):
    ans = float('-inf')
    while root:
        if root.data == key:
            return key
        if root.data < key:
            ans = root.data
            root = root.right
        else:
            root = root.left
    return ans if ans != float('-inf') else None

def ceil_bst(root, key):
    ans = float('inf')
    while root:
        if root.data == key:
            return key
        if root.data > key:
            ans = root.data
            root = root.left
        else:
            root = root.right
    return ans if ans != float('inf') else None
```

## 10. Code Explanation

- **Floor:** When we find a node  $\leq$  key, it's a candidate and we go right looking for a larger (but still  $\leq$  key) value. When we find a node  $>$  key, we go left.
- **Ceil:** Symmetric — when we find a node  $\geq$  key, it's a candidate and we go left looking for a smaller (but still  $\geq$  key) value.
- Both run in  $O(h)$  time and use  $O(1)$  space.

## 11. Complexity Analysis

| Operation   | Time   | Space  |
|-------------|--------|--------|
| Floor       | $O(h)$ | $O(1)$ |
| Ceil        | $O(h)$ | $O(1)$ |
| Successor   | $O(h)$ | $O(1)$ |
| Predecessor | $O(h)$ | $O(1)$ |

## 12. Common Patterns

- **Range query [L, R]:** Find floor(L), then iterate inorder until value  $> R$ .
- **Closest value:** Compute floor and ceil, pick the closest.
- **Nearest neighbor:** Same as closest value.

## 13. Common Mistakes

- Confusing floor with predecessor (strict vs non-strict).
- Initializing ans with wrong sentinel value.
- Forgetting to handle the case where no floor/ceil exists.

## 14. Edge Cases

| Case                 | Floor           | Ceil             |
|----------------------|-----------------|------------------|
| Empty tree           | -inf / null     | +inf / null      |
| Key smaller than all | -inf / null     | smallest element |
| Key larger than all  | largest element | +inf / null      |
| Key equals a node    | key             | key              |

## 15. Practice Problems

### Easy:

- [Floor in BST — GFG](#)
- [Ceil in BST — GFG](#)

### Medium:

- [Closest Binary Search Tree Value — LeetCode 270](#)
- [Find the Closest Element in BST — GFG](#)

## 16. Interview Explanation

"Floor is the largest value  $\leq$  key, ceil is the smallest value  $\geq$  key. We traverse the BST: for floor, whenever we see a value  $\leq$  key, we save it and try to go right for a better candidate. For ceil, whenever we see a value  $\geq$  key, we save it and try to go left. The time is  $O(h)$  and space is  $O(1)$ ."

## 17. Revision Notes

- Floor: candidate when node  $\leq$  key, go right.
- Ceil: candidate when node  $\geq$  key, go left.
- Strict vs non-strict: adjust comparison ( $\leq$  vs  $<$ ,  $\geq$  vs  $>$ ).

## 18. Final Cheat Sheet

| Aspect     | Detail                                                |
|------------|-------------------------------------------------------|
| Floor      | Largest $\leq$ key. Candidate when $\leq$ , go right. |
| Ceil       | Smallest $\geq$ key. Candidate when $\geq$ , go left. |
| Complexity | $O(h)$ time, $O(1)$ space                             |
| Edge       | Empty tree, key outside range                         |

# KTH SMALLEST IN BST

---

## 1. Overview

Find the  $k$ th smallest element in a BST (1-indexed). This leverages the fact that inorder traversal gives sorted order, so the  $k$ th element in inorder is the answer.

## 2. Intuition

**Finding the  $k$ th smallest person in a line sorted by height:** You count from the shortest. The  $k$ th person you count is the answer.

In a BST, inorder traversal visits elements in sorted order. So we can do an inorder traversal and stop when we've visited  $k$  elements.

With subtree size augmentation (each node stores the size of its subtree), we can find the  $k$ th element in  $O(h)$  without traversing the whole tree.

## 3. When to Use It

- Order statistics queries on a dynamic set.
- Problems: "kth smallest", "kth largest", "median in a stream".
- When you need to find the  $k$ th element repeatedly as the set changes.

## 4. When Not to Use It

- Static data → sort array and access index  $k-1$  directly  $O(n \log n)$  or use `nth_element`  $O(n)$ .
- Only one query → collect inorder and index (simpler).
- Frequent updates → use an order-statistic tree (augmented BST or Fenwick tree).

## 5. Core Concepts

### 5.1 Inorder Approach

Do inorder traversal. Keep a counter. When counter reaches  $k$ , return the current node's value.

This is  $O(n)$  worst-case even if  $k$  is small.

### 5.2 Augmented BST (Order-Statistic Tree)

Each node stores `size = 1 + size(left) + size(right)`. Then:

```
function kthSmallest(root, k):
    leftSize = size(root.left)
    if k == leftSize + 1: return root
    if k <= leftSize: return kthSmallest(root.left, k)
    else: return kthSmallest(root.right, k - leftSize - 1)
```

This is  $O(h)$  per query.

## 6. Step-by-Step Algorithm

### Inorder approach:

```
count = 0, ans = -1
function inorder(root, k):
    if root == null or count >= k: return
    inorder(root.left, k)
    count++
    if count == k: ans = root.data; return
    inorder(root.right, k)
```

### Augmented BST approach:

```
function kthSmallest(root, k):
    leftSize = root.left ? root.left.size : 0
    if k == leftSize + 1: return root.data
    if k <= leftSize: return kthSmallest(root.left, k)
    else: return kthSmallest(root.right, k - leftSize - 1)
```

## 7. Dry Run

### Tree (size in parentheses):

```
      50 (6)
     /  \
    30 (3) 70 (2)
   /  \   \
  20(1) 40(1) 80(1)
```

### Find k=4 (4th smallest):

| Step | Node | leftSize | k | Action                                             |
|------|------|----------|---|----------------------------------------------------|
| 1    | 50   | 3        | 4 | $4 > 3+1 \rightarrow$ go right, $k = 4-3-1 = 0$    |
| 2    | 70   | 0        | 0 | $k == 0$ ? No, formula: $k == \text{leftSize}+1$ ? |

Actually let's redo:

### Corrected: k=4

| Step | Node | leftSize | k | Action                                                            |
|------|------|----------|---|-------------------------------------------------------------------|
| 1    | 50   | 3        | 4 | $4 == 4$ ? No ( $3+1=4 \rightarrow$ yes!) $\rightarrow$ return 50 |

Wait,  $k=4$ ,  $\text{leftSize}=3$ , so  $k == \text{leftSize}+1 = 4$ . Return 50. ✓

Let's check  $k=2$ :

| Step | Node | leftSize | k | Action                                     |
|------|------|----------|---|--------------------------------------------|
| 1    | 50   | 3        | 2 | $2 \leq 3 \rightarrow$ go left             |
| 2    | 30   | 1        | 2 | $2 == 2? (1+1=2) \rightarrow$ return 30. ✓ |

Let's check  $k=5$ :

| Step | Node | leftSize | k | Action                                        |
|------|------|----------|---|-----------------------------------------------|
| 1    | 50   | 3        | 5 | $5 > 4 \rightarrow$ go right, $k = 5-3-1 = 1$ |
| 2    | 70   | 0        | 1 | $1 == 1? \text{yes} \rightarrow$ return 70. ✓ |

## 8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

template <typename T>
struct Node {
    T data;
    Node *left, *right;
    int size; // subtree size (augmented)
    Node(T val) : data(val), left(nullptr), right(nullptr), size(1) {}
};

// Update size after insert/delete
template <typename T>
int getSize(Node<T>* root) {
    return root ? root->size : 0;
}

template <typename T>
void updateSize(Node<T>* root) {
    if (root)
        root->size = 1 + getSize(root->left) + getSize(root->right);
}

// --- Approach 1: Inorder traversal (O(n)) ---
template <typename T>
T kthSmallestInorder(Node<T>* root, int k) {
    stack<Node<T>*> st;
    Node<T>* cur = root;
    int count = 0;
    while (cur || !st.empty()) {
        while (cur) {
            st.push(cur);
            cur = cur->left;
        }
```

```

    }
    cur = st.top(); st.pop();
    count++;
    if (count == k) return cur->data;
    cur = cur->right;
}
return T(); // not found
}

// --- Approach 2: Augmented BST (O(h)) ---
template <typename T>
T kthSmallest(Node<T>* root, int k) {
    if (!root) return T();
    int leftSize = getSize(root->left);
    if (k == leftSize + 1) return root->data;
    if (k <= leftSize) return kthSmallest(root->left, k);
    return kthSmallest(root->right, k - leftSize - 1);
}

```

## 9. Python Implementation

```

class Node:
    def __init__(self, val):
        self.data = val
        self.left = None
        self.right = None
        self.size = 1 # subtree size

def get_size(root):
    return root.size if root else 0

def update_size(root):
    if root:
        root.size = 1 + get_size(root.left) + get_size(root.right)

def kth_smallest(root, k):
    """Augmented BST approach. O(h)."""
    if not root:
        return None
    left_size = get_size(root.left)
    if k == left_size + 1:
        return root.data
    if k <= left_size:
        return kth_smallest(root.left, k)
    return kth_smallest(root.right, k - left_size - 1)

def kth_smallest_inorder(root, k):
    """Inorder traversal approach. O(n)."""
    stack, cur = [], root
    count = 0
    while cur or stack:

```

```
while cur:
    stack.append(cur)
    cur = cur.left
cur = stack.pop()
count += 1
if count == k:
    return cur.data
cur = cur.right
return None
```

## 10. Code Explanation

- **Inorder approach:** Simple, works without augmentation.  $O(n)$  time.
- **Augmented approach:** Requires maintaining **size** at each node.  $O(h)$  time. Size must be updated after every insert/delete.
- For kth largest, use `kthSmallest(root, n - k + 1)` or mirror the logic.

## 11. Complexity Analysis

| Approach              | Time   | Space  | Notes                     |
|-----------------------|--------|--------|---------------------------|
| Inorder               | $O(n)$ | $O(h)$ | Simple, no augmentation   |
| Augmented             | $O(h)$ | $O(h)$ | Requires size maintenance |
| Augmented + iterative | $O(h)$ | $O(1)$ | Possible with while loop  |

## 12. Common Patterns

- **Kth largest:** Use `kthSmallest(root, n - k + 1)` or traverse `right→root→left`.
- **Median:** Find  $n/2$ th and  $(n/2+1)$ th smallest.
- **Range queries augmented:** Store sum/min/max in subtree.

## 13. Common Mistakes

- Off-by-one:  $k$  is 1-indexed.
- Forgetting to update sizes after insert/delete in augmented BST.
- Not handling the case where  $k$  is out of range.

## 14. Edge Cases

| Case                      | Expected                     |
|---------------------------|------------------------------|
| $k = 1$                   | Minimum element              |
| $k = n$                   | Maximum element              |
| $k$ out of range $[1, n]$ | Error / sentinel             |
| Single node               | $k=1 \rightarrow$ node value |

## 15. Variations

- **Kth largest:** Mirror of kth smallest.
- **Kth smallest in BST with duplicates:** Use frequency count at each node.
- **Kth smallest in range [L, R]:** Combine with range query.

## 16. Practice Problems

### Easy:

- [Kth Smallest Element in a BST — LeetCode 230](#)
- [Kth largest element in BST — GFG](#)

### Medium:

- [Kth Smallest Element in a Sorted Matrix — LeetCode 378](#) (uses BST-like binary search)
- [Median in a stream of integers — GFG](#)

## 17. Interview Explanation

"For kth smallest in a BST, the simplest approach is inorder traversal — since inorder gives sorted order, we stop at the kth element. This is  $O(n)$ . For better performance, we can augment each node with its subtree size. Then we can compare  $k$  with the left subtree size to decide whether to go left, return the current node, or go right with adjusted  $k$ . This is  $O(h)$  per query."

## 18. Revision Notes

- Inorder = sorted. kth element in inorder = kth smallest.
- Augmented BST: store subtree size for  $O(h)$  queries.
- kth largest = kth smallest on mirrored tree or  $(n-k+1)$ .

## 19. Final Cheat Sheet

| Aspect     | Detail                                                                                               |
|------------|------------------------------------------------------------------------------------------------------|
| Approach 1 | Inorder traversal, stop at $k$ . $O(n)$                                                              |
| Approach 2 | Augmented BST with size. $O(h)$                                                                      |
| Formula    | $\text{leftSize} = \text{size}(\text{left})$ . If $k == \text{leftSize} + 1 \rightarrow \text{root}$ |
| Go left    | $k \leq \text{leftSize}$                                                                             |
| Go right   | $k > \text{leftSize} + 1$ , new $k = k - \text{leftSize} - 1$                                        |

## AVL TREE IDEA

### 1. Overview

An **AVL tree** is a self-balancing BST where the heights of the left and right subtrees of every node differ by at most 1 (the **balance factor**). This ensures the tree height is always  $O(\log n)$ , guaranteeing  $O(\log n)$  for all operations.

## 2. Intuition

**A tightrope walker with a balancing pole:** The pole must stay within a certain tilt. If it tilts too far left, you shift right. If it tilts too far right, you shift left.

In AVL trees, after every insertion or deletion, we check the **balance factor** ( $\text{height}(\text{left}) - \text{height}(\text{right})$ ) of each node. If it's outside  $[-1, 1]$ , we perform **rotations** to rebalance.

**Why rotations work:** They restructure the tree locally without breaking the BST property, and they reduce the height.

## 3. When to Use It

- You need guaranteed  $O(\log n)$  worst-case operations.
- Frequent insertions and deletions in a dynamic set.
- When worst-case performance matters (not just average).
- **Never** use a plain BST if data can be sorted or nearly sorted.

## 4. When Not to Use It

- Read-heavy workloads with few writes → Red-Black tree (fewer rotations).
- You need order statistics (kth smallest) → augmented AVL or Treap.
- You need range split/merge → Treap.
- Memory is very tight → AVL stores height per node (extra int).
- Simpler alternatives suffice: `std::set`, `std::map` (both Red-Black trees).

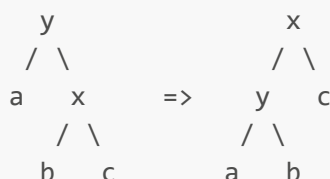
## 5. Core Concepts

### 5.1 Balance Factor

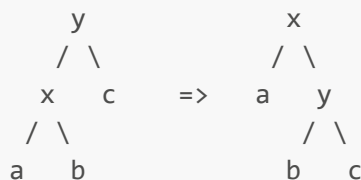
`balance = height(left) - height(right)`. Must be -1, 0, or +1.

### 5.2 Rotations

**Left Rotation (LL imbalance):** Right subtree is too heavy. Rotate node left.



**Right Rotation (RR imbalance):** Left subtree is too heavy. Rotate node right.



**Left-Right Rotation (LR imbalance):** Left child's right subtree is heavy. Left rotate left child, then right rotate node.

**Right-Left Rotation (RL imbalance):** Right child's left subtree is heavy. Right rotate right child, then left rotate node.

### 5.3 Height Maintenance

Each node stores its height. Height is updated after rotations and recursive calls.

## 6. Step-by-Step Algorithm

### Insert:

```

function insert(root, key):
    // 1. Standard BST insert
    if root == null: return new Node(key)
    if key < root.data: root.left = insert(root.left, key)
    else: root.right = insert(root.right, key)

    // 2. Update height
    root.height = 1 + max(height(root.left), height(root.right))

    // 3. Get balance factor
    balance = getBalance(root)

    // 4. Four cases:
    // Left Left
    if balance > 1 and key < root.left.data:
        return rightRotate(root)
    // Right Right
    if balance < -1 and key > root.right.data:
        return leftRotate(root)
    // Left Right
    if balance > 1 and key > root.left.data:
        root.left = leftRotate(root.left)
        return rightRotate(root)
    // Right Left
    if balance < -1 and key < root.right.data:
        root.right = rightRotate(root.right)
        return leftRotate(root)

    return root
  
```

## 7. Dry Run

**Insert 10, 20, 30 into empty AVL tree:**

**Insert 10:**

```
10 (balance = 0)
```

**Insert 20:**

```
10 (balance = -1, ok)
 \
  20
```

**Insert 30:**

```
10 (balance = -2, VIOLATION → RR case)
 \
  20
   \
    30
```

**After left rotation at 10:**

```
  20
 /  \
10   30  ✓ (balanced)
```

## 8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

template <typename T>
struct AVLNode {
    T data;
    AVLNode *left, *right;
    int height;
    AVLNode(T val) : data(val), left(nullptr), right(nullptr), height(1) {}
};

template <typename T>
int getHeight(AVLNode<T>* node) {
    return node ? node->height : 0;
}
```

```

}

template <typename T>
int getBalance(AVLNode<T>* node) {
    return node ? getHeight(node->left) - getHeight(node->right) : 0;
}

template <typename T>
AVLNode<T>* rightRotate(AVLNode<T>* y) {
    AVLNode<T>* x = y->left;
    AVLNode<T>* T2 = x->right;

    // Rotate
    x->right = y;
    y->left = T2;

    // Update heights
    y->height = 1 + max(getHeight(y->left), getHeight(y->right));
    x->height = 1 + max(getHeight(x->left), getHeight(x->right));

    return x; // new root
}

template <typename T>
AVLNode<T>* leftRotate(AVLNode<T>* x) {
    AVLNode<T>* y = x->right;
    AVLNode<T>* T2 = y->left;

    // Rotate
    y->left = x;
    x->right = T2;

    // Update heights
    x->height = 1 + max(getHeight(x->left), getHeight(x->right));
    y->height = 1 + max(getHeight(y->left), getHeight(y->right));

    return y; // new root
}

template <typename T>
AVLNode<T>* insertAVL(AVLNode<T>* root, T key) {
    // 1. Standard BST insert
    if (!root) return new AVLNode<T>(key);
    if (key < root->data)
        root->left = insertAVL(root->left, key);
    else if (key > root->data)
        root->right = insertAVL(root->right, key);
    else
        return root; // duplicate

    // 2. Update height
    root->height = 1 + max(getHeight(root->left), getHeight(root->right));

    // 3. Get balance factor

```

```

    int balance = getBalance(root);

    // 4. Rebalance if needed
    // Left Left
    if (balance > 1 && key < root->left->data)
        return rightRotate(root);
    // Right Right
    if (balance < -1 && key > root->right->data)
        return leftRotate(root);
    // Left Right
    if (balance > 1 && key > root->left->data) {
        root->left = leftRotate(root->left);
        return rightRotate(root);
    }
    // Right Left
    if (balance < -1 && key < root->right->data) {
        root->right = rightRotate(root->right);
        return leftRotate(root);
    }

    return root;
}

// Delete is similar: BST delete + rebalance
template <typename T>
AVLNode<T>* minValueNode(AVLNode<T>* root) {
    while (root && root->left) root = root->left;
    return root;
}

template <typename T>
AVLNode<T>* deleteAVL(AVLNode<T>* root, T key) {
    if (!root) return nullptr;

    if (key < root->data)
        root->left = deleteAVL(root->left, key);
    else if (key > root->data)
        root->right = deleteAVL(root->right, key);
    else {
        // Node with 0 or 1 child
        if (!root->left || !root->right) {
            AVLNode<T>* temp = root->left ? root->left : root->right;
            if (!temp) {
                temp = root;
                root = nullptr;
            } else {
                *root = *temp; // copy child data
            }
            delete temp;
        } else {
            // Two children
            AVLNode<T>* temp = minValueNode(root->right);
            root->data = temp->data;
            root->right = deleteAVL(root->right, temp->data);
        }
    }
}

```

```

    }
}

if (!root) return nullptr;

// Update height and rebalance
root->height = 1 + max(getHeight(root->left), getHeight(root->right));
int balance = getBalance(root);

// Left Left
if (balance > 1 && getBalance(root->left) >= 0)
    return rightRotate(root);
// Left Right
if (balance > 1 && getBalance(root->left) < 0) {
    root->left = leftRotate(root->left);
    return rightRotate(root);
}
// Right Right
if (balance < -1 && getBalance(root->right) <= 0)
    return leftRotate(root);
// Right Left
if (balance < -1 && getBalance(root->right) > 0) {
    root->right = rightRotate(root->right);
    return leftRotate(root);
}

return root;
}

```

## 9. Python Implementation

```

class AVLNode:
    def __init__(self, val):
        self.data = val
        self.left = None
        self.right = None
        self.height = 1

def get_height(node):
    return node.height if node else 0

def get_balance(node):
    return get_height(node.left) - get_height(node.right) if node else 0

def right_rotate(y):
    x = y.left
    t2 = x.right
    x.right = y
    y.left = t2
    y.height = 1 + max(get_height(y.left), get_height(y.right))
    x.height = 1 + max(get_height(x.left), get_height(x.right))

```

```

        return x

def left_rotate(x):
    y = x.right
    t2 = y.left
    y.left = x
    x.right = t2
    x.height = 1 + max(get_height(x.left), get_height(x.right))
    y.height = 1 + max(get_height(y.left), get_height(y.right))
    return y

def insert_avl(root, key):
    if not root:
        return AVLNode(key)
    if key < root.data:
        root.left = insert_avl(root.left, key)
    elif key > root.data:
        root.right = insert_avl(root.right, key)
    else:
        return root

    root.height = 1 + max(get_height(root.left), get_height(root.right))
    balance = get_balance(root)

    # Left Left
    if balance > 1 and key < root.left.data:
        return right_rotate(root)
    # Right Right
    if balance < -1 and key > root.right.data:
        return left_rotate(root)
    # Left Right
    if balance > 1 and key > root.left.data:
        root.left = left_rotate(root.left)
        return right_rotate(root)
    # Right Left
    if balance < -1 and key < root.right.data:
        root.right = right_rotate(root.right)
        return left_rotate(root)

    return root

```

## 10. Code Explanation

- **Insert:** Standard BST insert + height update + balance check + 4 rotation cases.
- **Delete:** Standard BST delete (with successor) + rebalance.
- **Rotations:** Left and right rotate are symmetric. They change the root of a subtree while preserving BST property.
- **Height update:** After every recursive call and rotation.

## 11. Complexity Analysis

| Operation | Time        | Space       |
|-----------|-------------|-------------|
| Insert    | $O(\log n)$ | $O(\log n)$ |
| Delete    | $O(\log n)$ | $O(\log n)$ |
| Search    | $O(\log n)$ | $O(\log n)$ |
| Space     | $O(n)$      | —           |

## 12. Common Patterns

- **Self-balancing BST:** Always maintain balance factor  $\leq 1$ .
- **Rotation identification:** The direction of imbalance determines which rotation(s) to apply.
- **LR and RL:** Two rotations needed; first rotation makes it LL or RR, second fixes it.

## 13. Common Mistakes

- Forgetting to update height after rotations.
- Not updating height before checking balance.
- Incorrect balance factor sign convention.
- Wrong rotation direction.
- Not handling the case where balance is 0 after rotation (still valid).

## 14. Edge Cases

| Case                   | Expected                          |
|------------------------|-----------------------------------|
| Insert into empty tree | New node returned                 |
| Insert sorted data     | Tree stays balanced               |
| Delete from leaf       | Leaf removed, rebalance ancestors |
| Delete root            | New root from rebalance           |

## 15. Variations

- **Top-down AVL:** Rebalance during insertion descent, not after. Avoids second pass.
- **AVL with parent pointers:** Allows iterative rebalancing.

## 16. Related Algorithms

- **Red-Black tree:** Less strict balancing, fewer rotations, more nodes.
- **Treap:** Randomized balancing, simpler code.
- **Splay tree:** Self-adjusting, no explicit balance info.

## 17. Practice Problems

### Medium:

- [AVL Tree Insertion — GFG](#)

- [AVL Tree Deletion — GFG](#)

**Hard:**

- [Balance a Binary Search Tree — LeetCode 1382](#)

## 18. Interview Explanation

"AVL tree is a self-balancing BST where the height difference between left and right subtrees is at most 1. After every insertion or deletion, we check the balance factor and perform rotations — left, right, left-right, or right-left — to restore balance. This guarantees  $O(\log n)$  height and therefore  $O(\log n)$  for all operations. The trade-off is the extra per-node height storage and the rebalancing work."

## 19. Revision Notes

- Balance factor =  $\text{height}(\text{left}) - \text{height}(\text{right}) \in \{-1, 0, 1\}$ .
- 4 rotation cases: LL, RR, LR, RL.
- Height updated after each operation.
- Guaranteed  $O(\log n)$  height.

## 20. Final Cheat Sheet

| Aspect     | Detail                             |
|------------|------------------------------------|
| When       | Guaranteed $O(\log n)$ worst-case  |
| Balance    | $\text{BF} \in \{-1, 0, 1\}$       |
| Rotations  | LL, RR, LR, RL                     |
| Height     | Stored per node, updated after ops |
| Complexity | All $O(\log n)$                    |

# RED-BLACK TREE IDEA

## 1. Overview

A **Red-Black Tree** is a self-balancing BST where each node has a color (red or black) and five properties are maintained to ensure approximate balance. The tree height is at most  $2 \times \log_2(n+1)$ , giving  $O(\log n)$  operations.

## 2. Intuition

**A traffic light system:** Red nodes are "stop" nodes that can't have red children. Black nodes are "go" nodes that create the tree's spine.

The red-black properties ensure:

- No path from root to leaf is more than twice as long as any other path.
- This gives a weaker guarantee than AVL ( $\text{height} \leq 2 \times \text{optimal}$ ) but with fewer rotations during insert/delete.

**Analogy:** Red-black tree is like a less strict AVL — it allows more imbalance but reduces the cost of rebalancing.

### 3. When to Use It

- You need  $O(\log n)$  worst-case operations with **frequent insertions/deletions**.
- You're using C++ `std::set` / `std::map` (they are red-black trees).
- You need a balanced BST with low rebalancing overhead.
- Write-heavy workloads where AVL's tighter balancing is costly.

### 4. When Not to Use It

- **Read-heavy workloads** → AVL is slightly faster (tighter balance).
- **Need order statistics** → augment with subtree size (possible but more complex than AVL).
- **Need range split/merge** → Treap.
- **Simple use case** → just use `std::set` (you don't need to implement it).

## 5. Core Concepts

### 5.1 Red-Black Properties

1. Every node is either red or black.
2. Root is always black.
3. Leaves (null pointers) are black.
4. Red nodes cannot have red children (no two consecutive reds).
5. Every path from root to leaf has the same number of black nodes (black-height).

### 5.2 Black-Height

The number of black nodes on any path from root to leaf. Property 5 ensures all paths have the same black-height.

### 5.3 Rotations

Same as AVL: left and right rotations. But the trigger is color violations, not height.

### 5.4 Recoloring

Often, violations can be fixed by changing colors without rotations. This is cheaper than rotations.

## 6. Step-by-Step Algorithm

### Insert (simplified):

```
Insert as standard BST, color new node RED.
While parent is RED:
    If uncle is RED:
        Recolor parent, uncle, grandparent
        Move up to grandparent
    Else (uncle is BLACK):
```

```

    Perform rotations and recolor
    Root is always BLACK

```

**Delete (complex — 6 cases):** Standard BST delete, then fix double-black violations.

## 7. Dry Run

**Insert 10, 20, 30, 15 into empty RB tree:**

**Insert 10:**

```

10(B)  // root is black

```

**Insert 20:**

```

10(B)
 \
 20(R)  // all fine

```

**Insert 30:**

```

10(B)      20(B)
 \    =>  /  \
 20(R)  10(R) 30(R)
        \
        30(R)  // uncle 10 is RED → recolor

```

**Insert 15:**

```

    20(B)
   /  \
 10(R) 30(R)
   \
  15(R)  // parent(10) red, uncle(30) red → recolor

```

After recolor:

```

    20(B)
   /  \
 10(B) 30(B)
   \
  15(R)  // parent is black, done

```

## 8. C++ Implementation

**Note:** Full RB tree implementation is ~200 lines. Here's a compact but complete version.

```
#include <bits/stdc++.h>
using namespace std;

enum Color { RED, BLACK };

template <typename T>
struct RBNode {
    T data;
    RBNode *left, *right, *parent;
    Color color;
    RBNode(T val) : data(val), left(nullptr), right(nullptr),
                  parent(nullptr), color(RED) {}
};

template <typename T>
class RBTree {
    RBNode<T>* root;

    void rotateLeft(RBNode<T>* x) {
        RBNode<T>* y = x->right;
        x->right = y->left;
        if (y->left) y->left->parent = x;
        y->parent = x->parent;
        if (!x->parent) root = y;
        else if (x == x->parent->left) x->parent->left = y;
        else x->parent->right = y;
        y->left = x;
        x->parent = y;
    }

    void rotateRight(RBNode<T>* y) {
        RBNode<T>* x = y->left;
        y->left = x->right;
        if (x->right) x->right->parent = y;
        x->parent = y->parent;
        if (!y->parent) root = x;
        else if (y == y->parent->left) y->parent->left = x;
        else y->parent->right = x;
        x->right = y;
        y->parent = x;
    }

    void fixViolation(RBNode<T>* z) {
        while (z->parent && z->parent->color == RED) {
            RBNode<T>* grandparent = z->parent->parent;
            if (z->parent == grandparent->left) {
                RBNode<T>* uncle = grandparent->right;
                // Case 1: uncle is RED → recolor
```

```

        if (uncle && uncle->color == RED) {
            grandparent->color = RED;
            z->parent->color = BLACK;
            uncle->color = BLACK;
            z = grandparent;
        } else {
            // Case 2: z is right child → left rotate
            if (z == z->parent->right) {
                z = z->parent;
                rotateLeft(z);
            }
            // Case 3: z is left child → right rotate
            z->parent->color = BLACK;
            grandparent->color = RED;
            rotateRight(grandparent);
        }
    } else {
        // Mirror cases
        RBNode<T>* uncle = grandparent->left;
        if (uncle && uncle->color == RED) {
            grandparent->color = RED;
            z->parent->color = BLACK;
            uncle->color = BLACK;
            z = grandparent;
        } else {
            if (z == z->parent->left) {
                z = z->parent;
                rotateRight(z);
            }
            z->parent->color = BLACK;
            grandparent->color = RED;
            rotateLeft(grandparent);
        }
    }
}

root->color = BLACK;
}

public:
    RBTree() : root(nullptr) {}

    void insert(T key) {
        RBNode<T>* z = new RBNode<T>(key);
        RBNode<T>* y = nullptr;
        RBNode<T>* x = root;

        // Standard BST insert
        while (x) {
            y = x;
            if (z->data < x->data) x = x->left;
            else if (z->data > x->data) x = x->right;
            else return; // duplicate
        }
        z->parent = y;
    }

```

```

        if (!y) root = z;
        else if (z->data < y->data) y->left = z;
        else y->right = z;

        fixViolation(z);
    }

    // Search, delete, inorder traversal omitted for brevity
    // (delete is complex with ~6 cases)
};

```

## 9. Python Implementation

```

class RBNode:
    def __init__(self, val):
        self.data = val
        self.left = None
        self.right = None
        self.parent = None
        self.color = 'RED' # 'RED' or 'BLACK'

class RBTree:
    def __init__(self):
        self.root = None

    def _rotate_left(self, x):
        y = x.right
        x.right = y.left
        if y.left:
            y.left.parent = x
        y.parent = x.parent
        if not x.parent:
            self.root = y
        elif x == x.parent.left:
            x.parent.left = y
        else:
            x.parent.right = y
        y.left = x
        x.parent = y

    def _rotate_right(self, y):
        x = y.left
        y.left = x.right
        if x.right:
            x.right.parent = y
        x.parent = y.parent
        if not y.parent:
            self.root = x
        elif y == y.parent.left:
            y.parent.left = x
        else:

```

```
        y.parent.right = x
    x.right = y
    y.parent = x

def _fix_violation(self, z):
    while z.parent and z.parent.color == 'RED':
        grandparent = z.parent.parent
        if z.parent == grandparent.left:
            uncle = grandparent.right
            if uncle and uncle.color == 'RED':
                grandparent.color = 'RED'
                z.parent.color = 'BLACK'
                uncle.color = 'BLACK'
                z = grandparent
            else:
                if z == z.parent.right:
                    z = z.parent
                    self._rotate_left(z)
                z.parent.color = 'BLACK'
                grandparent.color = 'RED'
                self._rotate_right(grandparent)
        else:
            uncle = grandparent.left
            if uncle and uncle.color == 'RED':
                grandparent.color = 'RED'
                z.parent.color = 'BLACK'
                uncle.color = 'BLACK'
                z = grandparent
            else:
                if z == z.parent.left:
                    z = z.parent
                    self._rotate_right(z)
                z.parent.color = 'BLACK'
                grandparent.color = 'RED'
                self._rotate_left(grandparent)
    self.root.color = 'BLACK'

def insert(self, key):
    z = RBNode(key)
    y, x = None, self.root
    while x:
        y = x
        if z.data < x.data:
            x = x.left
        elif z.data > x.data:
            x = x.right
        else:
            return
    z.parent = y
    if not y:
        self.root = z
    elif z.data < y.data:
        y.left = z
    else:
```

```
y.right = z
self._fix_violation(z)
```

## 10. Code Explanation

- **Insert:** Standard BST insert + **fixViolation** which handles 3 cases (uncle red, zig-zag, zig-zig).
- **Rotations:** Same as AVL but with parent pointers.
- **Colors:** Recoloring can fix violations without rotations.
- **Delete:** Much more complex (6 cases for double-black). Typically not asked in CP/placements to implement fully.

## 11. Complexity Analysis

| Operation | Time        | Space       |
|-----------|-------------|-------------|
| Insert    | $O(\log n)$ | $O(\log n)$ |
| Delete    | $O(\log n)$ | $O(\log n)$ |
| Search    | $O(\log n)$ | $O(1)$      |
| Space     | $O(n)$      | —           |

## 12. Common Patterns

- **std::set / std::map:** Both are red-black trees.
- **TreeMap / TreeSet in Java:** Red-black tree.
- **Linux kernel:** Uses red-black trees for memory management.

## 13. Common Mistakes

- Root must always be black.
- Red nodes cannot have red children.
- Null leaves are black.
- Delete is complex — don't implement from scratch in an interview unless asked.

## 14. Edge Cases

- Insert into empty tree → root black.
- Insert where uncle is both red and black → different cases.
- Delete root → handle carefully.

## 15. Variations

- **Left-leaning red-black tree:** Simpler variant (all red nodes are left children). Used in Robert Sedgwick's implementation.
- **AA tree:** A variation of red-black tree with simpler code.

## 16. Related Algorithms

- **AVL tree:** Tighter balance, more rotations.
- **Treap:** Randomized, simpler.
- **B-tree:** Generalization of red-black (2-3-4 tree).

## 17. Practice Problems

### Medium:

- [Red-Black Tree Insertion — GFG](#)
- [Implement Red-Black Tree — GFG](#)

### Hard:

- Red-Black Tree deletion (rarely asked in interviews).

## 18. Interview Explanation

"A red-black tree is a self-balancing BST with  $O(\log n)$  operations. Each node is red or black, and we maintain five properties — the most important being that no two reds are adjacent and every path has the same number of black nodes. This ensures the tree height is at most  $2 \times \log n$ . After insertions, we fix violations by recoloring or rotating. C++ sets and maps are red-black trees."

## 19. Revision Notes

- 5 properties: root black, no adjacent reds, equal black-height.
- Insert: recolor if uncle red, rotate if uncle black.
- Delete: complex, 6 cases → not expected to implement in interview.
- `std::set` / `std::map` are RB trees.

## 20. Final Cheat Sheet

| Aspect     | Detail                                                       |
|------------|--------------------------------------------------------------|
| When       | $O(\log n)$ with frequent writes                             |
| Properties | 5 rules (no adjacent red, equal black-height)                |
| Balancing  | Recolor + rotate                                             |
| Complexity | All $O(\log n)$                                              |
| Usage      | <code>std::set</code> , <code>std::map</code> , Java TreeMap |

# TREAP

## 1. Overview

A **Treap** (Tree + Heap) is a randomized BST where each node has a key (BST property) and a priority (heap property). The tree is balanced by randomly assigning priorities and maintaining the heap property through rotations.

## 2. Intuition

### A mix of BST and heap:

- Keys are organized as a BST (left < root < right).
- Priorities are organized as a heap (parent priority > child priority).

If you assign random priorities, the tree is balanced with high probability — like a randomly built BST.

**Analogy:** Imagine a group of people with different heights (keys) and randomly assigned ID numbers (priorities). You arrange them so that keys are in BST order but a taller (higher priority) person is always above shorter people. This naturally creates a balanced structure.

## 3. When to Use It

- **Simplest balanced BST to implement** (much fewer lines than AVL/RB).
- Need **split** and **merge** operations (unique to Treap among common BSTs).
- **Implicit treap** (treap on array indices) — used for range operations, reversal, etc.
- When you want good expected performance without complex balancing code.

## 4. When Not to Use It

- **Worst-case matters** (deterministic guarantee) — use AVL or RB tree.
- **Randomness is undesirable** (e.g., real-time systems).
- **Simple BST is enough** — no need for balancing.

## 5. Core Concepts

### 5.1 Key-Priority Structure

Each node has:

- **key** — maintains BST property.
- **prio** — random priority, maintains max-heap property.

### 5.2 Rotation

Same as AVL: left and right rotations. Rotations preserve BST property while fixing heap property.

### 5.3 Split and Merge (Crucial!)

**Split(root, key):** Splits the tree into two trees: one with keys  $\leq$  key, another with keys  $>$  key. Returns (left, right).

**Merge(left, right):** Merges two treaps where all keys in left < all keys in right. Returns the combined treap.

These operations make Treap extremely powerful for range operations.

## 6. Step-by-Step Algorithm

**Insert:**

```
function insert(root, key):
  if root == null: return new Node(key, random())
  if key < root.key:
    root.left = insert(root.left, key)
    if root.left.prio > root.prio:
      root = rightRotate(root)
  else:
    root.right = insert(root.right, key)
    if root.right.prio > root.prio:
      root = leftRotate(root)
  return root
```

### Split:

```
function split(root, key):
  if root == null: return (null, null)
  if root.key <= key:
    (left, right) = split(root.right, key)
    root.right = left
    return (root, right)
  else:
    (left, right) = split(root.left, key)
    root.left = right
    return (left, root)
```

### Merge:

```
function merge(left, right):
  if left == null: return right
  if right == null: return left
  if left.prio > right.prio:
    left.right = merge(left.right, right)
    return left
  else:
    right.left = merge(left, right.left)
    return right
```

## 7. Dry Run

### Insert 50, 30, 20 with random priorities:

Assume priorities: 50→75, 30→50, 20→90

### Insert 50 (prio=75):

```
50(75)
```

### Insert 30 (prio=50):

```
50(75)
 /
30(50)  // 30 < 50, goes left. 50.prio > 30.prio, ok
```

### Insert 20 (prio=90):

```
50(75)
 /
30(50)
 /
20(90)  // 20 < 30, goes left. 30.prio(50) < 20.prio(90) → rotate right at 30
```

After right rotation at 30:

```
50(75)
 /
20(90)
 \
 30(50)  // 20.prio(90) > 50.prio(75) → rotate right at 50
```

After right rotation at 50:

```
20(90)
 /  \
null 50(75)
      /
     30(50)
```

## 8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

template <typename T>
struct TreapNode {
    T key;
    int prio;
```

```

    TreapNode *left, *right;
    TreapNode(T k) : key(k), prio(rand()), left(nullptr), right(nullptr) {}
};

template <typename T>
TreapNode<T>* rotateRight(TreapNode<T>* y) {
    TreapNode<T>* x = y->left;
    TreapNode<T>* T2 = x->right;
    x->right = y;
    y->left = T2;
    return x;
}

template <typename T>
TreapNode<T>* rotateLeft(TreapNode<T>* x) {
    TreapNode<T>* y = x->right;
    TreapNode<T>* T2 = y->left;
    y->left = x;
    x->right = T2;
    return y;
}

// Insert using rotations
template <typename T>
TreapNode<T>* insertTreap(TreapNode<T>* root, T key) {
    if (!root) return new TreapNode<T>(key);
    if (key < root->key) {
        root->left = insertTreap(root->left, key);
        if (root->left->prio > root->prio)
            root = rotateRight(root);
    } else if (key > root->key) {
        root->right = insertTreap(root->right, key);
        if (root->right->prio > root->prio)
            root = rotateLeft(root);
    }
    return root;
}

// Split by key (keys ≤ key go to left, keys > key go to right)
template <typename T>
pair<TreapNode<T>*, TreapNode<T>*> split(TreapNode<T>* root, T key) {
    if (!root) return {nullptr, nullptr};
    if (root->key <= key) {
        auto [left, right] = split(root->right, key);
        root->right = left;
        return {root, right};
    } else {
        auto [left, right] = split(root->left, key);
        root->left = right;
        return {left, root};
    }
}

// Merge two treaps (all keys in left < all keys in right)

```

```

template <typename T>
TreapNode<T>* merge(TreapNode<T>* left, TreapNode<T>* right) {
    if (!left || !right) return left ? left : right;
    if (left->prio > right->prio) {
        left->right = merge(left->right, right);
        return left;
    } else {
        right->left = merge(left, right->left);
        return right;
    }
}

// Delete using merge: split into <key, ==key, >key, then merge <key and >key
template <typename T>
TreapNode<T>* eraseTreap(TreapNode<T>* root, T key) {
    auto [left, mid] = split(root, key - 1); // keys < key
    auto [target, right] = split(mid, key); // keys == key
    // delete target (or just ignore)
    return merge(left, right);
}

// Search
template <typename T>
bool searchTreap(TreapNode<T>* root, T key) {
    while (root) {
        if (root->key == key) return true;
        root = (key < root->key) ? root->left : root->right;
    }
    return false;
}

// Example usage
int main() {
    srand(time(0));
    TreapNode<int>* root = nullptr;
    for (int x : {5, 3, 7, 1, 9, 4, 6}) {
        root = insertTreap(root, x);
    }
    cout << searchTreap(root, 4) << "\n"; // 1
    root = eraseTreap(root, 4);
    cout << searchTreap(root, 4) << "\n"; // 0
    return 0;
}

```

## 9. Python Implementation

```

import random

class TreapNode:
    def __init__(self, key):
        self.key = key

```

```
        self.prio = random.randint(1, 1 << 30)
        self.left = None
        self.right = None

def rotate_right(y):
    x = y.left
    t2 = x.right
    x.right = y
    y.left = t2
    return x

def rotate_left(x):
    y = x.right
    t2 = y.left
    y.left = x
    x.right = t2
    return y

def insert_treap(root, key):
    if not root:
        return TreapNode(key)
    if key < root.key:
        root.left = insert_treap(root.left, key)
        if root.left.prio > root.prio:
            root = rotate_right(root)
    elif key > root.key:
        root.right = insert_treap(root.right, key)
        if root.right.prio > root.prio:
            root = rotate_left(root)
    return root

def split(root, key):
    """Split into ≤key and >key."""
    if not root:
        return (None, None)
    if root.key <= key:
        left, right = split(root.right, key)
        root.right = left
        return (root, right)
    else:
        left, right = split(root.left, key)
        root.left = right
        return (left, root)

def merge(left, right):
    if not left or not right:
        return left or right
    if left.prio > right.prio:
        left.right = merge(left.right, right)
        return left
    else:
        right.left = merge(left, right.left)
        return right
```

```
def erase_treap(root, key):
    left, mid = split(root, key - 1)
    target, right = split(mid, key)
    return merge(left, right)

def search_treap(root, key):
    while root:
        if root.key == key:
            return True
        root = root.left if key < root.key else root.right
    return False
```

## 10. Code Explanation

- **Insert with rotation:** Standard BST insert + heap property check → rotate if needed.
- **Split/merge:** Recursive. Split uses the key to decide which side goes where. Merge picks the higher priority root.
- **Delete via split:** Split into <key, ==key, >key, then merge key (dropping ==key).
- **Random priorities:** `rand()` or `random.randint()` ensures balanced expected height.

## 11. Complexity Analysis

| Operation | Expected Time | Worst-case | Space       |
|-----------|---------------|------------|-------------|
| Insert    | $O(\log n)$   | $O(n)$     | $O(\log n)$ |
| Delete    | $O(\log n)$   | $O(n)$     | $O(\log n)$ |
| Search    | $O(\log n)$   | $O(n)$     | $O(1)$      |
| Split     | $O(\log n)$   | $O(n)$     | $O(\log n)$ |
| Merge     | $O(\log n)$   | $O(n)$     | $O(\log n)$ |

Worst-case happens with extremely unlucky random priorities (probability  $\sim 0$ ).

## 12. Common Patterns

- **Implicit Treap (Cartesian Tree on indices):** Instead of key, use index. Apply split/merge for range operations.
- **Range reverse:** Implicit Treap with lazy propagation (swap flag).
- **Range add/query:** Implicit Treap with lazy sum.
- **Ordered set with split/merge:** Unique to Treap among balanced BSTs.

## 13. Common Mistakes

- Not seeding random ( `srand(time(0))`  in C++).
- Forgetting to handle duplicates.
- Incorrect split condition ( $\leq$  vs  $<$ ).
- Merge assumes all keys in left  $<$  all keys in right — violation breaks BST property.

## 14. Edge Cases

| Case                | Expected           |
|---------------------|--------------------|
| Split empty tree    | (null, null)       |
| Merge with null     | Returns the other  |
| Insert duplicate    | Ignored or handled |
| Delete non-existent | No change          |

## 15. Variations

- **Implicit Treap (Index-based):** Keys are array indices. Used for sequence operations.
- **Treap with subtree size:** Augmented for order statistics.
- **Persistent Treap:** All operations are persistent (see Persistent BST section).

## 16. Related Algorithms

- **AVL / Red-Black:** Deterministic balanced BSTs.
- **Skip list:** Randomized ordered data structure.
- **Splay tree:** Self-adjusting BST.

## 17. Practice Problems

### Medium:

- [Treap Insertion and Search — GFG](#)
- [Ordered Set — Codeforces](#) (Treap application)

### Hard:

- [Implicit Treap for Range Operations — Codeforces](#)
- [Array and Operations — Codeforces](#) (uses implicit treap)

## 18. Interview Explanation

"A Treap is a randomized BST where each node has a key and a random priority. It maintains both BST property on keys and max-heap property on priorities. Insertions use rotations to fix heap violations. The key advantage of Treap is the split and merge operations — we can split a treap into two by a key, and merge two treaps where all keys in one are smaller than the other. This makes range operations very easy. The expected height is  $O(\log n)$  with high probability."

## 19. Revision Notes

- Key  $\rightarrow$  BST, Priority  $\rightarrow$  Heap.
- Random priorities  $\rightarrow$  expected  $O(\log n)$  height.
- Split by key, merge two treaps.
- Delete: split into 3 parts, drop middle.
- Implicit treap for range operations.

## 20. Final Cheat Sheet

| Aspect     | Detail                                  |
|------------|-----------------------------------------|
| When       | Simplest balanced BST, need split/merge |
| Key        | BST property                            |
| Priority   | Max-heap property, random               |
| Core ops   | Split, merge (unique feature)           |
| Complexity | $O(\log n)$ expected                    |
| Edge       | Split/merge requires key ordering       |

## ORDERED SET

### 1. Overview

An **Ordered Set** (also called **Order-Statistic Tree**) is a balanced BST augmented with subtree sizes, supporting order statistics (kth smallest, rank of an element) in addition to standard BST operations.

### 2. Intuition

**A class roster with positions:** You can ask "who is 5th in the roster?" (kth smallest) and "what position is Alice?" (rank). As students join or leave, the positions update automatically.

In C++, `std::set` doesn't support order statistics. But we can build one using an augmented balanced BST (Treap, AVL, or RB tree with subtree sizes).

### 3. When to Use It

- You need **kth smallest** in a dynamic set (interleaved insert/delete/query).
- You need to find the **rank** (number of elements  $\leq$  key) of a value.
- Problems: "kth largest in a stream", "count of numbers in range", "order statistics".
- CP: When you can't use PBDS (`__gnu_pbds::tree`) and need to implement manually.

### 4. When Not to Use It

- Only need `lower_bound/upper_bound` → `std::set` suffices.
- Static data → sort + binary search / Fenwick tree.
- Need range updates (add to range) → Fenwick tree / segment tree.
- You can use PBDS in C++ (`#include <ext/pb_ds/assoc_container.hpp>`).

## 5. Core Concepts

### 5.1 Subtree Size Augmentation

Each node stores `size = 1 + size(left) + size(right)`. This is updated after every insert/delete.

## 5.2 Kth Smallest

Given  $k$  (1-indexed), compare  $k$  with left subtree size:

- If  $k == \text{leftSize} + 1 \rightarrow$  return current node.
- If  $k \leq \text{leftSize} \rightarrow$  go left.
- Else  $\rightarrow$  go right with  $k - \text{leftSize} - 1$ .

## 5.3 Rank of a Key

Number of elements  $\leq$  key. Traverse the tree:

- If  $\text{key} < \text{node.data} \rightarrow$  go left.
- If  $\text{key} > \text{node.data} \rightarrow$  add  $\text{leftSize} + 1$  to count, go right.
- If  $\text{key} == \text{node.data} \rightarrow$  return  $\text{leftSize} + 1 + \text{count}$ .

## 6. Step-by-Step Algorithm

**Kth smallest:**

```
function kth(root, k):
    leftSize = size(root.left)
    if k == leftSize + 1: return root.data
    if k <= leftSize: return kth(root.left, k)
    return kth(root.right, k - leftSize - 1)
```

**Rank of key:**

```
function rank(root, key):
    count = 0
    while root:
        if key < root.data:
            root = root.left
        else if key > root.data:
            count += size(root.left) + 1
            root = root.right
        else:
            return count + size(root.left) + 1
    return count // key not present, rank of insertion position
```

## 7. Dry Run

**Tree with sizes:**

```

      50 (6)
     /    \
   30 (3)  70 (2)
```

```

      /   \   \
    20(1) 40(1) 80(1)

```

### Kth smallest (k=4):

- At 50: leftSize=3, k=4 > 3+1? No, k=4 == 4 → return 50. ✓

### Rank of 40:

- At 50: 40 < 50 → go left.
- At 30: 40 > 30 → count += size(30.left)+1 = 1+1 = 2, go right.
- At 40: 40 == 40 → return 2 + size(40.left) + 1 = 2 + 0 + 1 = 3. ✓

## 8. C++ Implementation (using Treap as base)

```

#include <bits/stdc++.h>
using namespace std;

template <typename T>
struct OrderedSetNode {
    T key;
    int prio, size;
    OrderedSetNode *left, *right;
    OrderedSetNode(T k) : key(k), prio(rand()), size(1),
                        left(nullptr), right(nullptr) {}
};

template <typename T>
int getSize(OrderedSetNode<T>* node) {
    return node ? node->size : 0;
}

template <typename T>
void updateSize(OrderedSetNode<T>* node) {
    if (node)
        node->size = 1 + getSize(node->left) + getSize(node->right);
}

template <typename T>
OrderedSetNode<T>* rotateRight(OrderedSetNode<T>* y) {
    OrderedSetNode<T>* x = y->left;
    OrderedSetNode<T>* T2 = x->right;
    x->right = y;
    y->left = T2;
    updateSize(y);
    updateSize(x);
    return x;
}

template <typename T>
OrderedSetNode<T>* rotateLeft(OrderedSetNode<T>* x) {

```

```

    OrderedSetNode<T>* y = x->right;
    OrderedSetNode<T>* T2 = y->left;
    y->left = x;
    x->right = T2;
    updateSize(x);
    updateSize(y);
    return y;
}

template <typename T>
OrderedSetNode<T>* insert(OrderedSetNode<T>* root, T key) {
    if (!root) return new OrderedSetNode<T>(key);
    if (key < root->key) {
        root->left = insert(root->left, key);
        if (root->left->prio > root->prio)
            root = rotateRight(root);
    } else if (key > root->key) {
        root->right = insert(root->right, key);
        if (root->right->prio > root->prio)
            root = rotateLeft(root);
    }
    updateSize(root);
    return root;
}

template <typename T>
OrderedSetNode<T>* erase(OrderedSetNode<T>* root, T key) {
    if (!root) return nullptr;
    if (key == root->key) {
        // Merge children
        if (!root->left || !root->right) {
            OrderedSetNode<T>* temp = root->left ? root->left : root->right;
            delete root;
            return temp;
        }
        // Rotate down based on priority
        if (root->left->prio > root->right->prio) {
            root = rotateRight(root);
            root->right = erase(root->right, key);
        } else {
            root = rotateLeft(root);
            root->left = erase(root->left, key);
        }
    } else if (key < root->key) {
        root->left = erase(root->left, key);
    } else {
        root->right = erase(root->right, key);
    }
    updateSize(root);
    return root;
}

// Kth smallest (1-indexed)
template <typename T>

```

```

T kth(OrderedSetNode<T>* root, int k) {
    assert(root && k >= 1 && k <= root->size);
    int leftSize = getSize(root->left);
    if (k == leftSize + 1) return root->key;
    if (k <= leftSize) return kth(root->left, k);
    return kth(root->right, k - leftSize - 1);
}

// Rank of key (number of elements ≤ key)
template <typename T>
int rank(OrderedSetNode<T>* root, T key) {
    int ans = 0;
    while (root) {
        if (key < root->key) {
            root = root->left;
        } else if (key > root->key) {
            ans += getSize(root->left) + 1;
            root = root->right;
        } else {
            return ans + getSize(root->left) + 1;
        }
    }
    return ans; // key not present, rank of insertion position
}

// Count of elements in [L, R]
template <typename T>
int countInRange(OrderedSetNode<T>* root, T L, T R) {
    return rank(root, R) - rank(root, L - 1);
}

```

## 9. Python Implementation

```

import random

class OrderedSetNode:
    def __init__(self, key):
        self.key = key
        self.prio = random.randint(1, 1 << 30)
        self.size = 1
        self.left = None
        self.right = None

def get_size(node):
    return node.size if node else 0

def update_size(node):
    if node:
        node.size = 1 + get_size(node.left) + get_size(node.right)

def rotate_right(y):

```

```
x = y.left
t2 = x.right
x.right = y
y.left = t2
update_size(y)
update_size(x)
return x

def rotate_left(x):
    y = x.right
    t2 = y.left
    y.left = x
    x.right = t2
    update_size(x)
    update_size(y)
    return y

def insert(root, key):
    if not root:
        return OrderedSetNode(key)
    if key < root.key:
        root.left = insert(root.left, key)
        if root.left.prio > root.prio:
            root = rotate_right(root)
    elif key > root.key:
        root.right = insert(root.right, key)
        if root.right.prio > root.prio:
            root = rotate_left(root)
    update_size(root)
    return root

def kth(root, k):
    """1-indexed kth smallest."""
    assert root and 1 <= k <= root.size
    left_size = get_size(root.left)
    if k == left_size + 1:
        return root.key
    if k <= left_size:
        return kth(root.left, k)
    return kth(root.right, k - left_size - 1)

def rank(root, key):
    """Number of elements <= key."""
    ans = 0
    while root:
        if key < root.key:
            root = root.left
        elif key > root.key:
            ans += get_size(root.left) + 1
            root = root.right
        else:
            return ans + get_size(root.left) + 1
    return ans
```

## 10. Code Explanation

- **Augmentation:** `size` field updated after every insert/delete.
- **Kth smallest:** Uses left subtree size to decide direction.
- **Rank:** Traverses the tree, accumulating counts from left subtrees.
- **Range count:** `rank(R) - rank(L-1)`.
- Based on Treap for simplicity; can be built on any balanced BST.

## 11. Complexity Analysis

| Operation    | Time        | Space       |
|--------------|-------------|-------------|
| Insert       | $O(\log n)$ | $O(\log n)$ |
| Delete       | $O(\log n)$ | $O(\log n)$ |
| Kth smallest | $O(\log n)$ | $O(\log n)$ |
| Rank         | $O(\log n)$ | $O(1)$      |
| Range count  | $O(\log n)$ | $O(1)$      |

## 12. Common Patterns

- **Kth largest:** `kth(root, size - k + 1)`.
- **Count in range [L, R]:** `rank(R) - rank(L-1)`.
- **Median:** `kth(root, size/2 + 1)`.
- **Ordered set with duplicates:** Store frequency at each node.

## 13. Common Mistakes

- Forgetting to update sizes after rotations.
- Off-by-one in kth (1-indexed vs 0-indexed).
- Not handling the case where key is not present in rank.
- Using `rank` incorrectly for range count (needs `L-1`, not `L`).

## 14. Edge Cases

| Case                                   | Expected                                        |
|----------------------------------------|-------------------------------------------------|
| Empty tree                             | <code>kth</code> → error, <code>rank</code> → 0 |
| <code>k = 1</code>                     | Minimum element                                 |
| <code>k = size</code>                  | Maximum element                                 |
| Rank of element not present            | Insertion position                              |
| Range count with <code>L &gt; R</code> | 0                                               |

## 15. Variations

- **PBDS in C++:** `__gnu_pbds::tree<int, null_type, less<int>, rb_tree_tag, tree_order_statistics_node_update>` — built-in order statistics.
- **Fenwick tree for order statistics:** Works for small key ranges (compressible).
- **Segment tree for order statistics:** For static ranges.

## 16. Related Algorithms

- **Fenwick tree:** Order statistics for bounded key ranges.
- **Segment tree:** Range queries with point updates.
- **std::set:** No order statistics.

## 17. Practice Problems

### Easy:

- [Kth Smallest Element in a BST — LeetCode 230](#)
- [Ordered Set — CSES](#) (if available)

### Medium:

- [Ordered Set — Codeforces](#)
- [Find Median from Data Stream — LeetCode 295](#)
- [Count of Smaller Numbers After Self — LeetCode 315](#)

### Hard:

- [Reverse Pairs — LeetCode 493](#)
- [Range Sum Query with Updates — LeetCode 307](#) (Fenwick alternative)

## 18. Interview Explanation

"An ordered set is a BST augmented with subtree sizes. It supports standard BST operations plus order statistics — finding the kth smallest element and finding the rank of a key. The kth smallest works by comparing k with the left subtree size: if they match, we're at the right node; if k is smaller, go left; otherwise go right with adjusted k. Rank works by traversing and accumulating left subtree sizes. Both run in  $O(\log n)$ ."

## 19. Revision Notes

- Augment BST with subtree **size**.
- Kth: compare k with leftSize.
- Rank: traverse, add leftSize+1 when going right.
- Range count:  $\text{rank}(R) - \text{rank}(L-1)$ .
- Update size after every structural change.

## 20. Final Cheat Sheet

| Aspect | Detail                               |
|--------|--------------------------------------|
| When   | Need order statistics on dynamic set |

| Aspect       | Detail                          |
|--------------|---------------------------------|
| Augmentation | Subtree size                    |
| Kth          | Compare k with leftSize         |
| Rank         | Traverse, accumulate leftSize+1 |
| Complexity   | $O(\log n)$ per operation       |

# SPLAY TREE

## 1. Overview

A **Splay Tree** is a self-adjusting BST where every operation (search, insert, delete) moves the accessed node to the root via **splaying** (a series of rotations). Recently accessed elements are near the root, giving amortized  $O(\log n)$  performance.

## 2. Intuition

**A self-organizing bookshelf:** The more you read a book, the closer to the front you keep it. If you're reading a book, you splay it to the front. Next time, it's right there.

In a splay tree, every time you access a node, you **splay** it to the root. This means:

- Frequently accessed elements are near the top (fast access).
- The tree self-organizes based on access patterns.
- No explicit balance information is stored (no height, no color, no priority).

## 3. When to Use It

- **Access patterns are skewed** — some elements accessed much more frequently.
- **Temporal locality** — recently accessed elements are likely to be accessed again.
- **Simple implementation** — no balance info, no colors, no priorities.
- **Amortized guarantees are acceptable** — individual operations can be  $O(n)$  but amortized is  $O(\log n)$ .

## 4. When Not to Use It

- **Worst-case latency matters** — a single access can be  $O(n)$  (though rare).
- **Real-time systems** — unpredictable worst-case time.
- **You need  $O(\log n)$  per operation** — use AVL or RB tree.
- **Tree is rarely accessed** — no benefit from splaying.

## 5. Core Concepts

### 5.1 Splaying

A series of rotations that bring a node to the root. Three cases:

- **Zig (node is child of root):** Single rotation.

- **Zig-Zig (node and parent are both left or both right children):** Two rotations (rotate parent, then node).
- **Zig-Zag (node is left, parent is right or vice versa):** Two rotations in opposite directions.

## 5.2 Amortized Analysis

The splay operation has amortized  $O(\log n)$  time. This is proven using the **potential method** with the rank of a node =  $\log_2(\text{size of its subtree})$ .

## 5.3 Self-Adjusting

No balance information is stored. The tree reorganizes itself based on access patterns.

# 6. Step-by-Step Algorithm

### Splay:

```
function splay(root, key):
    if root == null or root.key == key: return root
    if key < root.key:
        if root.left == null: return root    // not found
        if key < root.left.key:              // Zig-Zig (left-left)
            root.left.left = splay(root.left.left, key)
            root = rightRotate(root)
        else if key > root.left.key:         // Zig-Zag (left-right)
            root.left.right = splay(root.left.right, key)
            if root.left.right != null:
                root.left = leftRotate(root.left)
        // Zig case
        if root.left != null:
            return rightRotate(root)
    return root
else: // mirror for right
    ...
```

**Search:** Splay the key to root. If root's key matches, found.

**Insert:** Splay the parent position, then attach new node.

**Delete:** Splay the key to root, then merge left and right subtrees.

## 7. Dry Run

### Splay 30 in:

```

      50
     /  \
    30   70
```

```

/
20

```

**Zig-Zig (30 is left child of 30's parent? No. Let's redo.)**

**Splay 20 in:**

```

    50
   /  \
  30   70
 /
20

```

**Step 1:**  $20 < 50$ ,  $20 < 30 \rightarrow$  Zig-Zig (left-left):

- Right rotate at 30:

```

    50
   /  \
  20   70
   \
   30

```

- Right rotate at 50:

```

  20
   \
   50
  /  \
 30   70

```

**Result:** 20 is at root. 

## 8. C++ Implementation

```

#include <bits/stdc++.h>
using namespace std;

template <typename T>
struct SplayNode {
    T data;
    SplayNode *left, *right;
    SplayNode(T val) : data(val), left(nullptr), right(nullptr) {}
};

```

```

template <typename T>
SplayNode<T>* rightRotate(SplayNode<T>* y) {
    SplayNode<T>* x = y->left;
    y->left = x->right;
    x->right = y;
    return x;
}

template <typename T>
SplayNode<T>* leftRotate(SplayNode<T>* x) {
    SplayNode<T>* y = x->right;
    x->right = y->left;
    y->left = x;
    return y;
}

// Splay the node with given key to root
template <typename T>
SplayNode<T>* splay(SplayNode<T>* root, T key) {
    if (!root || root->data == key) return root;

    if (key < root->data) {
        if (!root->left) return root;

        // Zig-Zig (left-left)
        if (key < root->left->data) {
            root->left->left = splay(root->left->left, key);
            root = rightRotate(root);
        }
        // Zig-Zag (left-right)
        else if (key > root->left->data) {
            root->left->right = splay(root->left->right, key);
            if (root->left->right)
                root->left = leftRotate(root->left);
        }

        // Zig case
        if (root->left)
            return rightRotate(root);
        return root;
    } else {
        if (!root->right) return root;

        // Zag-Zag (right-right)
        if (key > root->right->data) {
            root->right->right = splay(root->right->right, key);
            root = leftRotate(root);
        }
        // Zag-Zig (right-left)
        else if (key < root->right->data) {
            root->right->left = splay(root->right->left, key);
            if (root->right->left)
                root->right = rightRotate(root->right);
        }
    }
}

```

```

        // Zag case
        if (root->right)
            return leftRotate(root);
        return root;
    }
}

template <typename T>
SplayNode<T>* searchSplay(SplayNode<T>* root, T key) {
    return splay(root, key);
}

template <typename T>
SplayNode<T>* insertSplay(SplayNode<T>* root, T key) {
    if (!root) return new SplayNode<T>(key);

    // Splay the parent position
    root = splay(root, key);

    // If key already exists, return
    if (root->data == key) return root;

    SplayNode<T>* newNode = new SplayNode<T>(key);
    if (key < root->data) {
        newNode->right = root;
        newNode->left = root->left;
        root->left = nullptr;
    } else {
        newNode->left = root;
        newNode->right = root->right;
        root->right = nullptr;
    }
    return newNode;
}

template <typename T>
SplayNode<T>* deleteSplay(SplayNode<T>* root, T key) {
    if (!root) return nullptr;

    root = splay(root, key);
    if (root->data != key) return root; // not found

    // Merge left and right subtrees
    if (!root->left) {
        SplayNode<T>* temp = root->right;
        delete root;
        return temp;
    }
    if (!root->right) {
        SplayNode<T>* temp = root->left;
        delete root;
        return temp;
    }
}

```

```

    // Splay the max of left subtree to root of left
    SplayNode<T>* leftSubtree = root->left;
    SplayNode<T>* rightSubtree = root->right;
    delete root;

    leftSubtree = splay(leftSubtree, key); // will splay the max
    leftSubtree->right = rightSubtree;
    return leftSubtree;
}

```

## 9. Python Implementation

```

class SplayNode:
    def __init__(self, val):
        self.data = val
        self.left = None
        self.right = None

def right_rotate(y):
    x = y.left
    y.left = x.right
    x.right = y
    return x

def left_rotate(x):
    y = x.right
    x.right = y.left
    y.left = x
    return y

def splay(root, key):
    if not root or root.data == key:
        return root

    if key < root.data:
        if not root.left:
            return root
        if key < root.left.data:
            root.left.left = splay(root.left.left, key)
            root = right_rotate(root)
        elif key > root.left.data:
            root.left.right = splay(root.left.right, key)
            if root.left.right:
                root.left = left_rotate(root.left)
        if root.left:
            return right_rotate(root)
        return root
    else:
        if not root.right:
            return root

```

```

        if key > root.right.data:
            root.right.right = splay(root.right.right, key)
            root = left_rotate(root)
        elif key < root.right.data:
            root.right.left = splay(root.right.left, key)
            if root.right.left:
                root.right = right_rotate(root.right)
        if root.right:
            return left_rotate(root)
        return root

def insert_splay(root, key):
    if not root:
        return SplayNode(key)
    root = splay(root, key)
    if root.data == key:
        return root
    new_node = SplayNode(key)
    if key < root.data:
        new_node.right = root
        new_node.left = root.left
        root.left = None
    else:
        new_node.left = root
        new_node.right = root.right
        root.right = None
    return new_node

def delete_splay(root, key):
    if not root:
        return None
    root = splay(root, key)
    if root.data != key:
        return root
    if not root.left:
        return root.right
    if not root.right:
        return root.left
    left_sub = root.left
    right_sub = root.right
    left_sub = splay(left_sub, key) # splay max to root
    left_sub.right = right_sub
    return left_sub

```

## 10. Code Explanation

- **Splay:** Recursive function that brings the target node to the root. Handles zig, zig-zig, zig-zag cases.
- **Search:** Splay the key; if root's data matches, found.
- **Insert:** Splay the parent position, then attach new node as root with old root as child.
- **Delete:** Splay the key to root, then merge left and right subtrees (splay max of left to root of left, then attach right).

## 11. Complexity Analysis

| Operation | Amortized Time | Worst-case | Space       |
|-----------|----------------|------------|-------------|
| Search    | $O(\log n)$    | $O(n)$     | $O(1)$      |
| Insert    | $O(\log n)$    | $O(n)$     | $O(\log n)$ |
| Delete    | $O(\log n)$    | $O(n)$     | $O(\log n)$ |

## 12. Common Patterns

- **Cache-friendly:** Recently accessed elements are near root.
- **Dequeues, queues:** Splay trees support efficient split/merge.
- **Link-cut trees:** Used in dynamic graph connectivity (splay tree is the building block).

## 13. Common Mistakes

- Recursive splay can overflow the stack.
- Forgetting to handle the case where key is not found during splay.
- Incorrect zig-zag vs zig-zig identification.

## 14. Edge Cases

| Case                | Expected                        |
|---------------------|---------------------------------|
| Splay on empty tree | null                            |
| Splay on root       | root unchanged                  |
| Splay on leaf       | leaf becomes root               |
| Sequential access   | Tree reorganizes for efficiency |

## 15. Variations

- **Top-down splay tree:** Iterative splaying during descent (more efficient).
- **Semi-splay tree:** Splay only partway to root.
- **Link-cut tree:** Uses splay trees for dynamic tree connectivity.

## 16. Related Algorithms

- **AVL / Red-Black:** Deterministic  $O(\log n)$ , no amortization.
- **Treap:** Randomized, simpler.
- **Scapegoat tree:** Rebuilds when imbalance exceeds threshold.

## 17. Practice Problems

**Medium:**

- [Implement Splay Tree — GFG](#)

**Hard:**

- Dynamic connectivity problems (link-cut trees).
- Cache-efficient data structures.

## 18. Interview Explanation

"A splay tree is a self-adjusting BST where every accessed node is moved to the root through a process called splaying. This involves a series of rotations — zig, zig-zig, and zig-zag. Frequently accessed elements stay near the root, giving fast access. Operations have amortized  $O(\log n)$  time, though individual operations can be  $O(n)$  in worst case. No balance information is stored, making the implementation relatively simple."

## 19. Revision Notes

- Every operation splays the accessed node to root.
- 3 cases: zig, zig-zig, zig-zag.
- No balance info stored.
- Amortized  $O(\log n)$  per operation.
- Good for skewed access patterns.

## 20. Final Cheat Sheet

| Aspect          | Detail                                          |
|-----------------|-------------------------------------------------|
| When            | Skewed access patterns, amortized guarantees OK |
| How             | Splay accessed node to root                     |
| Cases           | Zig, Zig-Zig, Zig-Zag                           |
| Complexity      | Amortized $O(\log n)$ , worst $O(n)$            |
| No balance info | No height, color, or priority                   |

# PERSISTENT BST

## 1. Overview

A **Persistent BST** (also called **Persistent Treap** or **Persistent Segment Tree** in some contexts) is a BST that supports **versioning** — every insert or delete creates a **new version** of the tree while preserving the old versions. You can query any previous version.

## 2. Intuition

**Git for data structures:** Every time you make a change, you get a new snapshot. You can go back to any previous snapshot and query it.

**Time travel:** You can ask "what was the tree like after the 3rd insertion?" or "what was the kth smallest at that time?"

In a persistent BST, we don't modify existing nodes. Instead, we create new nodes along the path that changes. Unchanged subtrees are shared between versions (**path copying**).

### 3. When to Use It

- You need to query **historical versions** of a data structure.
- Problems: "Kth smallest over time", "range queries with updates".
- **Persistent Segment Tree** (a persistent BST on a fixed array).
- **Offline algorithms** that need to roll back to previous states.
- **Functional programming** where immutability is required.

### 4. When Not to Use It

- You only need the current version — use a regular BST.
- Memory is tight — persistent versions store  $O(\log n)$  new nodes per operation.
- You need to modify the current version frequently — the overhead of creating new nodes is significant.
- Simpler alternatives: just store the history of operations and replay.

## 5. Core Concepts

### 5.1 Path Copying

When a node is modified, we create a new copy of that node and all its ancestors (up to the root). The unchanged subtrees are shared (pointers to old nodes).

- **Insert:** Creates  $O(\log n)$  new nodes (the path from root to insertion point).
- **Delete:** Similarly creates  $O(\log n)$  new nodes.

### 5.2 Version Array

An array (or vector) stores the root pointer of each version. `version[i]` gives the root of the tree after the  $i$ -th operation.

### 5.3 Immutability

Old nodes are never modified. This allows safe concurrent access and makes reasoning about the code easier.

## 6. Step-by-Step Algorithm

**Persistent Insert (in Treap, using split/merge):**

```
function insert(root, key):
    left, mid = split(root, key)    // keys ≤ key
    mid = merge(mid, new Node(key)) // insert key
    return merge(left, mid)         // merge back
```

Since split and merge create new nodes along the path, this is already persistent if we copy nodes during split/merge.

## 7. Dry Run

**Insert 50 into empty tree → version 1:**

50

**Insert 30 → version 2 (new nodes in red, shared in black):**

```

50(shared)      30(new)
 /              / \
30(new)  →    30  50

```

Wait, in path copying, we copy the path. Let me redo:

**Version 1 (root1):**

50

**Insert 30 into version 1 → version 2 (root2):**

```

50(new)      30(new)
 /           / \
30(new)     30  50

```

Actually, path copying: when we insert 30, we create a new 50 (copy of old 50) and a new 30. The old 50 remains unchanged.

```

Version 1:      Version 2:
  50             50'(new)
                 /
                30(new)

```

## 8. C++ Implementation (Persistent Treap)

```

#include <bits/stdc++.h>
using namespace std;

template <typename T>
struct PSTNode {
    T key;
    int prio;

```

```

PSTNode *left, *right;
PSTNode(T k) : key(k), prio(rand()), left(nullptr), right(nullptr) {}
// Copy constructor for persistence
PSTNode(PSTNode* other) : key(other->key), prio(other->prio),
                        left(other->left), right(other->right) {}
};

// Copy-on-write: create a new node if we're about to modify it
template <typename T>
PSTNode<T>* clone(PSTNode<T>* node) {
    return node ? new PSTNode<T>(node) : nullptr;
}

template <typename T>
pair<PSTNode<T>*, PSTNode<T>*> split(PSTNode<T>* root, T key) {
    if (!root) return {nullptr, nullptr};
    PSTNode<T>* newRoot = clone(root); // copy node for persistence
    if (root->key <= key) {
        auto [left, right] = split(root->right, key);
        newRoot->right = left;
        return {newRoot, right};
    } else {
        auto [left, right] = split(root->left, key);
        newRoot->left = right;
        return {left, newRoot};
    }
}

template <typename T>
PSTNode<T>* merge(PSTNode<T>* left, PSTNode<T>* right) {
    if (!left || !right) return left ? left : right;
    if (left->prio > right->prio) {
        PSTNode<T>* newLeft = clone(left);
        newLeft->right = merge(left->right, right);
        return newLeft;
    } else {
        PSTNode<T>* newRight = clone(right);
        newRight->left = merge(left, right->left);
        return newRight;
    }
}

// Persistent insert: returns new root without modifying old tree
template <typename T>
PSTNode<T>* insertPersistent(PSTNode<T>* root, T key) {
    auto [left, right] = split(root, key);
    PSTNode<T>* newNode = new PSTNode<T>(key);
    return merge(merge(left, newNode), right);
}

// Persistent erase: returns new root
template <typename T>
PSTNode<T>* erasePersistent(PSTNode<T>* root, T key) {
    auto [left, mid] = split(root, key - 1);

```

```

    auto [target, right] = split(mid, key);
    // target contains keys == key, we drop it
    return merge(left, right);
}

// Search in a specific version
template <typename T>
bool searchPersistent(PSTNode<T>* root, T key) {
    while (root) {
        if (root->key == key) return true;
        root = (key < root->key) ? root->left : root->right;
    }
    return false;
}

// Example: multiple versions
int main() {
    srand(time(0));
    vector<PSTNode<int>*> versions;
    versions.push_back(nullptr); // version 0: empty

    versions.push_back(insertPersistent(versions[0], 50)); // v1
    versions.push_back(insertPersistent(versions[1], 30)); // v2
    versions.push_back(insertPersistent(versions[2], 70)); // v3

    // Query version 2
    cout << searchPersistent(versions[2], 70) << "\n"; // 0 (70 not in v2)
    cout << searchPersistent(versions[3], 70) << "\n"; // 1 (70 in v3)

    return 0;
}

```

## 9. Python Implementation

```

import random
import copy

class PSTNode:
    def __init__(self, key, left=None, right=None):
        self.key = key
        self.prio = random.randint(1, 1 << 30)
        self.left = left
        self.right = right

def clone(node):
    """Return a shallow copy (new node, same children)."""
    if not node:
        return None
    new = PSTNode.__new__(PSTNode)
    new.key = node.key
    new.prio = node.prio

```

```

    new.left = node.left
    new.right = node.right
    return new

def split(root, key):
    """Split into ≤key and >key. Returns copies of modified nodes."""
    if not root:
        return (None, None)
    new_root = clone(root)
    if root.key <= key:
        left, right = split(root.right, key)
        new_root.right = left
        return (new_root, right)
    else:
        left, right = split(root.left, key)
        new_root.left = right
        return (left, new_root)

def merge(left, right):
    if not left or not right:
        return left or right
    if left.prio > right.prio:
        new_left = clone(left)
        new_left.right = merge(left.right, right)
        return new_left
    else:
        new_right = clone(right)
        new_right.left = merge(left, right.left)
        return new_right

def insert_persistent(root, key):
    left, right = split(root, key)
    new_node = PSTNode(key)
    return merge(merge(left, new_node), right)

def erase_persistent(root, key):
    left, mid = split(root, key - 1)
    target, right = split(mid, key)
    return merge(left, right)

def search_persistent(root, key):
    while root:
        if root.key == key:
            return True
        root = root.left if key < root.key else root.right
    return False

# Example: multiple versions
versions = [None] # v0: empty
versions.append(insert_persistent(versions[0], 50)) # v1
versions.append(insert_persistent(versions[1], 30)) # v2
versions.append(insert_persistent(versions[2], 70)) # v3

```

```
print(search_persistent(versions[2], 70)) # False
print(search_persistent(versions[3], 70)) # True
```

## 10. Code Explanation

- **Clone:** Creates a new node with the same data and children. The old node is not modified.
- **Split/Merge:** Uses `clone` before modifying any node. This creates new nodes along the path being modified.
- **Insert:** `split(root, key) → merge(merge(left, newNode), right)`.
- **Delete:** `split` twice to isolate the key, then merge the outer parts.
- **Versioning:** Each operation returns a new root. Store roots in an array.

## 11. Complexity Analysis

| Operation         | Time                    | Space (new nodes)                |
|-------------------|-------------------------|----------------------------------|
| Insert            | $O(\log n)$             | $O(\log n)$                      |
| Delete            | $O(\log n)$             | $O(\log n)$                      |
| Search            | $O(\log n)$             | $O(1)$                           |
| Space per version | $O(n + k \cdot \log n)$ | where $k$ = number of operations |

## 12. Common Patterns

- **Persistent Segment Tree:** Most common persistent data structure in CP. Used for K-th number in range, etc.
- **Persistent Union-Find:** DSU with rollback.
- **Persistent Array:** Array with versioning (using persistent segment tree).

## 13. Common Mistakes

- Forgetting to clone nodes before modification.
- Memory leaks (old nodes are not garbage collected in C++).
- Not understanding that old versions are not modified — iterators/pointers to old nodes remain valid.
- Using `new` without `delete` in C++ (memory grows unbounded).

## 14. Edge Cases

| Case                    | Expected                              |
|-------------------------|---------------------------------------|
| Query version 0 (empty) | null / empty                          |
| Insert duplicate        | Depends on handling                   |
| Delete from old version | Leaves old version unchanged          |
| Many versions           | Memory grows linearly with operations |

## 15. Variations

- **Partial Persistence:** Can query any version but only modify the latest.
- **Full Persistence:** Can query and modify any version (creates branches).
- **Confluent Persistence:** Can merge two versions (creates a DAG, not a tree).
- **Persistent Segment Tree:** Most important in CP — range queries on array versions.

## 16. Related Algorithms

- **Persistent Segment Tree:** Fixed array with range queries on historical versions.
- **Persistent DSU:** Union-Find with rollback.
- **Functional Data Structures:** Immutable data structures in functional programming.

## 17. Practice Problems

### Medium:

- [Persistent Segment Tree — CSES Range Queries](#)
- [K-th Number in Range — SPOJ](#) (uses persistent segment tree)

### Hard:

- [Persistent Treap — Codeforces](#)
- [Persistent Bookcase — Codeforces 707D](#)

## 18. Interview Explanation

"A persistent BST preserves all previous versions of the tree. When we insert or delete, we create new nodes along the path that changes — this is called path copying. Unchanged subtrees are shared between versions. This way, we can query any historical version in  $O(\log n)$  time. The space cost is  $O(\log n)$  per operation because each operation only creates nodes along one path from root to leaf."

## 19. Revision Notes

- Path copying: copy only modified nodes.
- Shared unchanged subtrees.
- Each version has its own root.
- $O(\log n)$  time per operation,  $O(\log n)$  space per operation.
- Merge/split based persistence (Treap) is easier than AVL/RB.

## 20. Final Cheat Sheet

| Aspect     | Detail                                           |
|------------|--------------------------------------------------|
| When       | Need historical queries, versioning              |
| How        | Path copying — clone nodes on modification       |
| Core ops   | Split, merge (with clone)                        |
| Complexity | $O(\log n)$ time, $O(\log n)$ space per op       |
| Memory     | Total $O(n + k \cdot \log n)$ for $k$ operations |

| Aspect | Detail                     |
|--------|----------------------------|
| Edge   | Old versions are immutable |

## REVISION CHECKLIST

| Topic                 | Key Idea                            | Complexity            | CP/Interview Relevance |
|-----------------------|-------------------------------------|-----------------------|------------------------|
| <b>BST</b>            | left < root < right                 | $O(h)$                | ★★★★★                  |
| <b>Search</b>         | Compare → left/right                | $O(h)$                | ★★★★★                  |
| <b>Insert</b>         | Insert at leaf                      | $O(h)$                | ★★★★★                  |
| <b>Delete</b>         | 3 cases: leaf, 1 child, 2 children  | $O(h)$                | ★★★★★                  |
| <b>Inorder</b>        | left → root → right = sorted        | $O(n)$                | ★★★★★                  |
| <b>Floor/Ceil</b>     | Traverse, track candidates          | $O(h)$                | ★★★★                   |
| <b>Kth Smallest</b>   | Compare with leftSize               | $O(h)$ augmented      | ★★★★★                  |
| <b>AVL</b>            | $BF \in \{-1, 0, 1\}$ , 4 rotations | $O(\log n)$           | ★★★                    |
| <b>Red-Black</b>      | 5 properties, recolor + rotate      | $O(\log n)$           | ★★                     |
| <b>Treap</b>          | Random priority, heap property      | $O(\log n)$ exp.      | ★★★★                   |
| <b>Ordered Set</b>    | BST + subtree size                  | $O(\log n)$           | ★★★★★                  |
| <b>Splay Tree</b>     | Splay to root on access             | $O(\log n)$ amortized | ★★                     |
| <b>Persistent BST</b> | Path copying, versioning            | $O(\log n)$           | ★★★                    |

End of Guide — Happy Coding! 🚀